

Sorghum-Rye Biomass and Flux Analysis

Jared Flater

2026-04-07

Table of contents

1	Libraries	1
2	Theme	2
3	Data loading	3
4	Figure 1: Precipitation and GDD plots with historical envelopes	3
5	Figure 2: Soil moisture and temperature from LiCOR survey chamber	7
6	Figure 3: Sorghum and Sorghum + WCC biomass	14
7	Figure 4: Mean daily and cumulative nitrous oxide emissions	14
8	plot-level cumulative range accross all sorghum sorghum + WCC and corn for the 2 years	20
9	Daily flux details	23
10	Row vs. Interrow flux comparisons	35
11	Figure 5: Nitrate Leaching and Precipitation	40
12	Figure S_NEW: Stacked Bar — NH4/NO3 Proportions Over Time	49
13	Table 3: Annual Nitrogen Losses	51
14	Table S2: Cover Crop Biomass	79
15	Save Tables and Figures	80

1 Libraries

```
library(tidyverse)
library(readxl)
library(lubridate)
library(patchwork)
options(width = 80)
```

2 Theme

```
# Custom theme for consistency
theme_publication <- function(base_size = 12) {
  theme_minimal(base_size = base_size, base_family = "Helvetica") +
    theme(
      # Backgrounds
      panel.background = element_rect(fill = "#ffffff", color = NA),
      plot.background = element_rect(fill = "#ffffff", color = NA),
      strip.background = element_rect(fill = alpha("#A5CAD2", 0.2)),

      # Grid
      panel.grid.major = element_line(color = alpha("#B7B5B3", 0.2)),
      panel.grid.minor = element_blank(),
      panel.spacing = unit(2, "lines"),
      panel.border = element_rect(color = "black", fill = NA, linewidth = 1),

      # Text & Axis
      text = element_text(color = "#000000ff"),
      axis.text = element_text(color = "#000000ff", size = base_size * 0.8),
      axis.title = element_text(size = base_size),
      axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1),
      axis.line = element_line(color = "black", linewidth = 0.5),
      axis.ticks = element_line(color = "black", linewidth = 0.5),

      # Legend
      legend.position = "top",
      legend.justification = "center",
      legend.direction = "horizontal",
      legend.box = "horizontal",
      legend.margin = margin(b = 1, unit = "pt"),
      legend.box.margin = margin(b = 1, unit = "pt"),
      legend.text = element_text(size = base_size * 0.9),
      legend.title = element_text(size = base_size),
      legend.key.width = unit(0.8, "cm"),
      legend.key.height = unit(0.3, "cm")
    )
}

# Treatment colors
treatment_colors <- c(
  "Sorghum" = "#D8D97AFF",
  "Sorghum + Rye" = "#95C36EFF",
  "Sorghum + WCC" = "#95C36EFF",
  "Corn" = "#74C8C3FF",
  "Soy" = "#0A2E57FF"
)

scale_color_treatments <- function() scale_color_manual(values = treatment_colors)
scale_fill_treatments <- function() scale_fill_manual(values = treatment_colors)
```

3 Data loading

```
rain <- read_xlsx("data/rain/historical_rain.xlsx")
gdd <- read_xlsx("data/rain/historical_rain.xlsx")
flux_data <- read_csv("data/cleaned_flux_data.csv") |>
  mutate(date = as.Date(date))
```

4 Figure 1: Precipitation and GDD plots with historical envelopes

```
# Precipitation and GDD plots with historical envelopes
# Import historical rain and gdd data

gdd <- gdd %>%
  select(day, doy, gdd_50_86) %>%
  mutate(
    date = as.Date(day),
    year = year(date),
  ) %>%
  group_by(year) %>%
  arrange(date) %>%
  mutate(cumulative_gdd = cumsum(gdd_50_86))

gdd <- gdd %>%
  mutate(doy_date = as.Date(doy - 1, origin = "2000-01-01"))

rain <- rain %>%
  select(day, doy, precipmm) %>%
  mutate(
    date = as.Date(day),
    year = year(date),
  ) %>%
  group_by(year) %>%
  arrange(date) %>%
  mutate(cumulative_precip = cumsum(precipmm))

rain <- rain %>%
  mutate(doy_date = as.Date(doy - 1, origin = "2000-01-01"))

# Calculate historical envelope for RAIN (excluding 2023 and 2024)
hist_env_rain <- rain |>
  filter(!(year %in% c(2023, 2024))) |>
  group_by(doy_date) |>
  summarize(
    min_cum = min(cumulative_precip, na.rm = TRUE),
    max_cum = max(cumulative_precip, na.rm = TRUE)
  )

# Calculate historical envelope for GDD (excluding 2023 and 2024)
hist_env_gdd <- gdd |>
  filter(!(year %in% c(2023, 2024))) |>
  group_by(doy_date) |>
  summarize(
```

```

    min_cum = min(cumulative_gdd, na.rm = TRUE),
    max_cum = max(cumulative_gdd, na.rm = TRUE)
  )

# Calculate historical mean for both rain and GDD
mean_cum_rain <- rain |>
  filter(!(year %in% c(2023, 2024))) |>
  group_by(doy_date) |>
  summarize(cumulative_precip = mean(cumulative_precip, na.rm = TRUE)) |>
  mutate(year = "Mean") |>
  filter(doy_date != "2000-12-31")

mean_cum_gdd <- gdd |>
  filter(!(year %in% c(2023, 2024))) |>
  group_by(doy_date) |>
  summarize(cumulative_gdd = mean(cumulative_gdd, na.rm = TRUE)) |>
  mutate(year = "Mean") |>
  filter(doy_date != "2000-12-31")

# Combine highlighted years with mean
rain_highlight <- rain |>
  filter(year %in% c(2023, 2024)) |>
  mutate(year = as.character(year)) |>
  bind_rows(mean_cum_rain)

gdd_highlight <- gdd |>
  filter(year %in% c(2023, 2024)) |>
  mutate(year = as.character(year)) |>
  bind_rows(mean_cum_gdd)

# Rain plot
rain_plot_v2 <- ggplot() +
  geom_ribbon(
    data = hist_env_rain,
    aes(x = doy_date, ymin = min_cum, ymax = max_cum),
    fill = "gray80", alpha = 0.6
  ) +
  geom_line(
    data = rain_highlight,
    aes(x = doy_date, y = cumulative_precip, color = year, linetype = year),
    linewidth = 0.6
  ) +
  scale_color_manual(
    name = "Year",
    values = c("2023" = "#E41A1C", "2024" = "#377EB8", "Mean" = "black"),
    labels = c("2023" = "2023", "2024" = "2024", "Mean" = "Mean (Historical)")
  ) +
  scale_linetype_manual(
    name = "Year",
    values = c("2023" = "solid", "2024" = "dashed", "Mean" = "dotdash"),
    labels = c("2023" = "2023", "2024" = "2024", "Mean" = "Mean (Historical)")
  ) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b") +

```

```

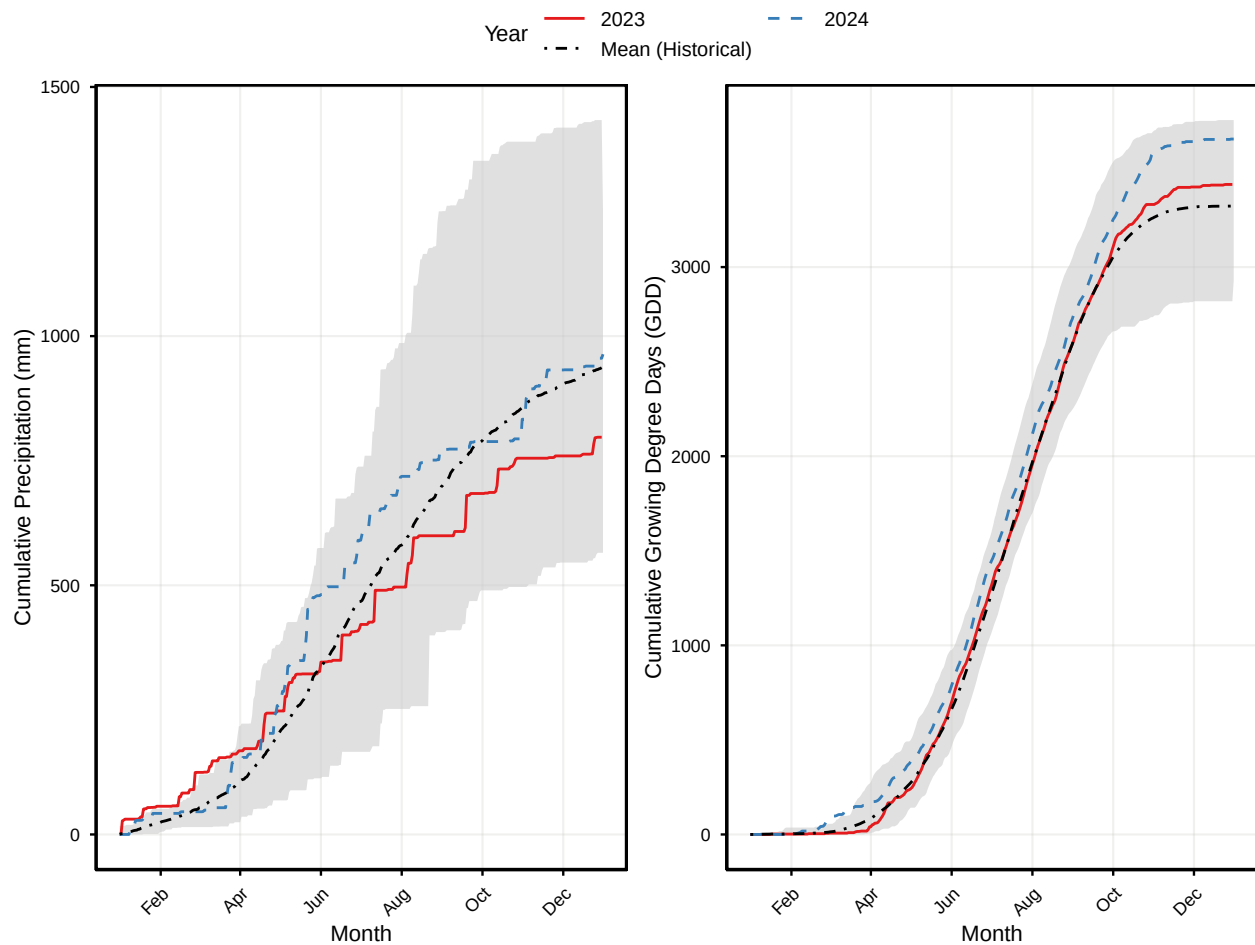
labs(x = "Month", y = "Cumulative Precipitation (mm)") +
theme_publication(base_size = 10)

# GDD plot
gdd_plot_v2 <- ggplot() +
  geom_ribbon(
    data = hist_env_gdd,
    aes(x = doy_date, ymin = min_cum, ymax = max_cum),
    fill = "gray80", alpha = 0.6
  ) +
  geom_line(
    data = gdd_highlight,
    aes(x = doy_date, y = cumulative_gdd, color = year, linetype = year),
    linewidth = 0.6
  ) +
  scale_color_manual(
    name = "Year",
    values = c("2023" = "#E41A1C", "2024" = "#377EB8", "Mean" = "black"),
    labels = c("2023" = "2023", "2024" = "2024", "Mean" = "Mean (Historical)")
  ) +
  scale_linetype_manual(
    name = "Year",
    values = c("2023" = "solid", "2024" = "dashed", "Mean" = "dotdash"),
    labels = c("2023" = "2023", "2024" = "2024", "Mean" = "Mean (Historical)")
  ) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b") +
  labs(x = "Month", y = "Cumulative Growing Degree Days (GDD)") +
  theme_publication(base_size = 10)

# Combine into pdplot with shared legend
six_panel_plot <- (rain_plot_v2 | gdd_plot_v2) +
  plot_layout(guides = "collect") &
  theme_publication(base_size = 10) &
  theme(
    legend.position = "top",
    plot.margin = margin(t = 3, r = 4, b = 2, l = 4, unit = "pt")
  ) &
  guides(
    color = guide_legend(nrow = 2, byrow = TRUE),
    linetype = guide_legend(nrow = 2, byrow = TRUE)
  )

ggsave("figures/figure1_weather.png", six_panel_plot,
       width = 84, height = 105, dpi = 600, bg = "white", units = "mm")
six_panel_plot

```



```
# Report date ranges used for historical means (excludes 2023, 2024)
hist_range_rain <- rain |>
  filter(!(year %in% c(2023, 2024))) |>
  ungroup() |>
  summarize(
    start_date = min(year, na.rm = TRUE),
    end_date   = max(year, na.rm = TRUE)
  )

hist_range_gdd <- gdd |>
  filter(!(year %in% c(2023, 2024))) |>
  ungroup() |>
  summarize(
    start_date = min(year, na.rm = TRUE),
    end_date   = max(year, na.rm = TRUE)
  )

cat("Historical mean period (rain):", hist_range_rain$start_date, "to", hist_range_rain$end_date, "\n")

Historical mean period (rain): 1988 to 2022

cat("Historical mean period (GDD):", hist_range_gdd$start_date, "to", hist_range_gdd$end_date, "\n")

Historical mean period (GDD): 1988 to 2022
```

5 Figure 2: Soil moisture and temperature from LiCOR survey chamber

```
# Soil moisture and temperature 4-panel plot

# Calculate summary statistics
moist_temp <- flux_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment)) |>
  group_by(Treatment, year, date) |>
  summarise(
    mean_soil_moisture = mean(swc_2_mean, na.rm = TRUE),
    mean_soil_temp = mean(ts_2_mean, na.rm = TRUE),
    .groups = "drop"
  )

# Per-date t-tests between treatments
raw_soil <- flux_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    year = year(date)
  )

get_date_pvals <- function(df, value_col) {
  df |>
    group_by(year, date) |>
    summarise(
      p = {
        vals <- .data[[value_col]]
        trt <- Treatment
        has_two <- length(unique(na.omit(trt))) == 2
        counts <- table(trt[!is.na(vals)])
        ok <- has_two && all(counts >= 1) && length(na.omit(vals)) >= 2
        if (ok) {
          out <- tryCatch(t.test(vals ~ trt)$p.value, error = function(e) NA_real_)
          out
        } else NA_real_
      },
      .groups = "drop"
    ) |>
    filter(!is.na(p) & p < 0.05)
}

pvals_moist <- get_date_pvals(raw_soil, "swc_2_mean")
pvals_temp <- get_date_pvals(raw_soil, "ts_2_mean")

# Split data by year
moist_temp_2023 <- filter(moist_temp, year == 2023)
moist_temp_2024 <- filter(moist_temp, year == 2024)

# Calculate global y-axis limits
global_moisture_limits <- range(moist_temp$mean_soil_moisture, na.rm = TRUE)
```

```

global_temp_limits <- range(moist_temp$mean_soil_temp, na.rm = TRUE)

global_moisture_limits[2] <- global_moisture_limits[2] * 1.12
global_temp_limits[2] <- 40

moisture_breaks <- pretty(global_moisture_limits, n = 5)
temp_breaks <- pretty(global_temp_limits, n = 5)

# Star positions for moisture
moisture_stars <- moist_temp |>
  group_by(year, date) |>
  summarise(y_star = max(mean_soil_moisture, na.rm = TRUE) * 1.05, .groups = "drop") |>
  inner_join(pvals_moist, by = c("year", "date")) |>
  filter(is.finite(y_star))

moisture_segments <- moisture_stars |>
  distinct(year, date) |>
  left_join(moist_temp, by = c("year", "date")) |>
  group_by(year, date) |>
  slice_max(order_by = mean_soil_moisture, n = 1, with_ties = FALSE) |>
  mutate(
    x = date - 2,
    xend = date + 2,
    y = 0.5
  ) |>
  ungroup()

# Star positions for temperature
temp_stars <- moist_temp |>
  group_by(year, date) |>
  summarise(y_star = max(mean_soil_temp, na.rm = TRUE) * 1.05, .groups = "drop") |>
  inner_join(pvals_temp, by = c("year", "date")) |>
  filter(is.finite(y_star))

temp_segments <- temp_stars |>
  distinct(year, date) |>
  left_join(moist_temp, by = c("year", "date")) |>
  group_by(year, date) |>
  slice_max(order_by = mean_soil_temp, n = 1, with_ties = FALSE) |>
  mutate(
    x = date - 2,
    xend = date + 2,
    y = 40
  ) |>
  ungroup()

# 2023 Soil Moisture (bottom left)
p1 <- ggplot(moist_temp_2023, aes(x = date, y = mean_soil_moisture, color = Treatment, linetype = Treatment)) +
  geom_line(linewidth = 0.6) +
  geom_point(size = 1.5) +
  scale_color_manual(values = treatment_colors) +
  scale_linetype_manual(values = c("Sorghum" = "dashed", "Sorghum + WCC" = "solid")) +
  scale_shape_manual(values = c("Sorghum" = 16, "Sorghum + WCC" = 15)) +

```

```

scale_x_date(date_breaks = "2 months", date_labels = "%b",
             limits = c(as.Date("2023-01-01"), as.Date("2023-12-31"))) +
scale_y_continuous(limits = global_moisture_limits, breaks = moisture_breaks,
                  expand = expansion(mult = c(0.02, 0.02))) +
labs(x = "Month", y = expression("Volumetric Soil Moisture (cm"3*" "cm"-3)"),
     color = "Treatment", linetype = "Treatment", shape = "Treatment") +
geom_hline(yintercept = 0.5, color = "black", linewidth = 7) +
geom_segment(
  data = moisture_segments |> filter(year == 2023),
  aes(x = x, xend = xend, y = y, yend = y, color = Treatment),
  linewidth = 7, inherit.aes = FALSE, show.legend = FALSE
) +
annotate("text", x = as.Date("2023-01-15"), y = 0.47,
         label = "c", size = 4, color = "black", fontface = "bold") +
theme_publication(base_size = 10) +
theme(legend.position = "none")

# 2024 Soil Moisture (bottom right)
p2 <- ggplot(moist_temp_2024, aes(x = date, y = mean_soil_moisture, color = Treatment, linetype = Treatment)) +
  geom_line(linewidth = 0.6) +
  geom_point(size = 1.5) +
  scale_color_manual(values = treatment_colors) +
  scale_linetype_manual(values = c("Sorghum" = "dashed", "Sorghum + WCC" = "solid")) +
  scale_shape_manual(values = c("Sorghum" = 16, "Sorghum + WCC" = 15)) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b",
               limits = c(as.Date("2024-01-01"), as.Date("2024-12-31"))) +
  scale_y_continuous(limits = global_moisture_limits, breaks = moisture_breaks,
                    expand = expansion(mult = c(0.02, 0.02))) +
  labs(x = "Month", y = "", color = "Treatment", linetype = "Treatment", shape = "Treatment") +
  geom_hline(yintercept = 0.5, color = "black", linewidth = 7) +
  geom_segment(
    data = moisture_segments |> filter(year == 2024),
    aes(x = x, xend = xend, y = y, yend = y, color = Treatment),
    linewidth = 7, inherit.aes = FALSE, show.legend = FALSE
  ) +
  annotate("text", x = as.Date("2024-01-15"), y = 0.47,
         label = "d", size = 4, color = "black", fontface = "bold") +
  theme_publication(base_size = 10) +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    axis.line.y = element_blank(),
    legend.position = "none"
  )

# 2023 Soil Temperature (top left)
p3 <- ggplot(moist_temp_2023, aes(x = date, y = mean_soil_temp, color = Treatment, linetype = Treatment)) +
  geom_line(linewidth = 0.6) +
  geom_point(size = 1.5) +
  scale_color_manual(values = treatment_colors) +
  scale_linetype_manual(values = c("Sorghum" = "dashed", "Sorghum + WCC" = "solid")) +
  scale_shape_manual(values = c("Sorghum" = 16, "Sorghum + WCC" = 15)) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b",

```

```

        limits = c(as.Date("2023-01-01"), as.Date("2023-12-31"))) +
scale_y_continuous(limits = global_temp_limits, breaks = temp_breaks,
                    expand = expansion(mult = c(0.05, 0.05))) +
labs(x = "", y = "Soil Temperature (°C)", color = "Treatment", linetype = "Treatment", shape = "Treatment") +
geom_hline(yintercept = 40, color = "black", linewidth = 7) +
geom_segment(
  data = temp_segments |> filter(year == 2023),
  aes(x = x, xend = xend, y = y, yend = y, color = Treatment),
  linewidth = 7, inherit.aes = FALSE, show.legend = FALSE
) +
annotate("text", x = as.Date("2023-01-15"), y = 37,
          label = "a", size = 4, color = "black", fontface = "bold") +
theme_publication(base_size = 10) +
theme(
  axis.text.x = element_blank(),
  axis.ticks.x = element_blank(),
  axis.line.x = element_blank(),
  legend.position = "none"
)

# 2024 Soil Temperature (top right)
p4 <- ggplot(moist_temp_2024, aes(x = date, y = mean_soil_temp, color = Treatment, linetype = Treatment)) +
  geom_line(linewidth = 0.6) +
  geom_point(size = 1.5) +
  scale_color_manual(values = treatment_colors) +
  scale_linetype_manual(values = c("Sorghum" = "dashed", "Sorghum + WCC" = "solid")) +
  scale_shape_manual(values = c("Sorghum" = 16, "Sorghum + WCC" = 15)) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b",
               limits = c(as.Date("2024-01-01"), as.Date("2024-12-31"))) +
  scale_y_continuous(limits = global_temp_limits, breaks = temp_breaks,
                     expand = expansion(mult = c(0.05, 0.05))) +
  labs(x = "", y = "", color = "Treatment", linetype = "Treatment", shape = "Treatment") +
  geom_hline(yintercept = 40, color = "black", linewidth = 7) +
  geom_segment(
    data = temp_segments |> filter(year == 2024),
    aes(x = x, xend = xend, y = y, yend = y, color = Treatment),
    linewidth = 7, inherit.aes = FALSE, show.legend = FALSE
  ) +
  annotate("text", x = as.Date("2024-01-15"), y = 37,
          label = "b", size = 4, color = "black", fontface = "bold") +
  theme_publication(base_size = 10) +
  theme(
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank(),
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    axis.line = element_blank(),
    legend.position = "none"
  )

# Combine panels
soil_4panel <- (p3 | p4) / (p1 | p2) +
  patchwork::plot_layout(guides = "collect", heights = c(1, 1)) &

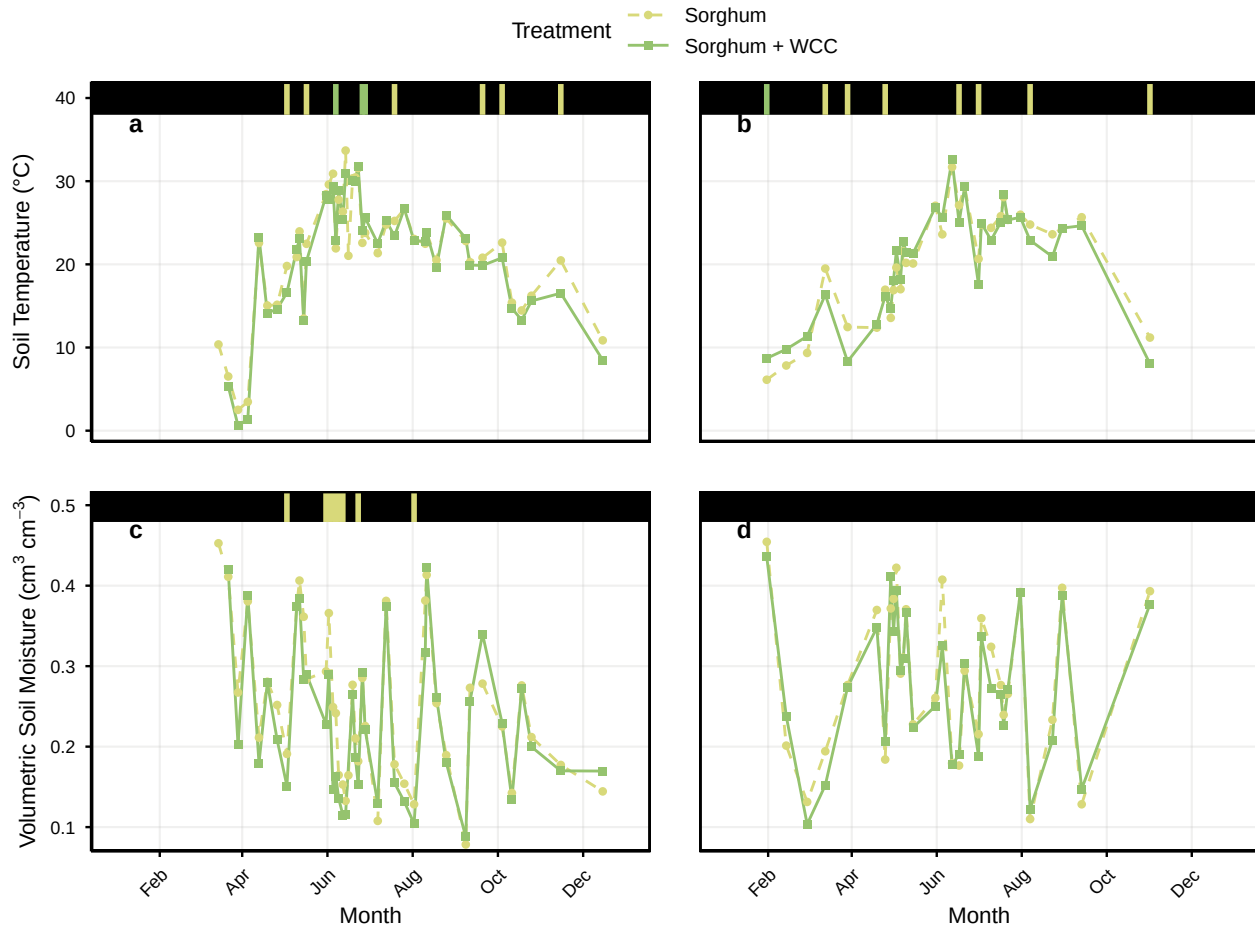
```

```

theme(legend.position = "top") &
guides(
  color = guide_legend(nrow = 2, byrow = TRUE),
  linetype = guide_legend(nrow = 2, byrow = TRUE),
  shape = guide_legend(nrow = 2, byrow = TRUE)
)

```

soil_4panel



```

# Print the range in soil temperature and moisture

```

```

cat("Soil Moisture Range:", round(global_moisture_limits[1], 3), "to", round(global_moisture_limits[2],

```

```

Soil Moisture Range: 0.078 to 0.509 cm3 cm-3

```

```

cat("Soil Temperature Range:", round(global_temp_limits[1], 2), "to", round(global_temp_limits[2], 2),

```

```

Soil Temperature Range: 0.61 to 40 °C

```

```

ggsave("figures/figure2_soil_moisture_temperature.png", soil_4panel,
  width = 174, height = 174, dpi = 600, bg = "white", units = "mm")

```

```

# ---- Extract + summarize SIGNIFICANT date-level treatment differences (Figure 2) ----

```

```

dir.create("data/processed", recursive = TRUE, showWarnings = FALSE)

```

```

# Helper: build wide table and join p-values

```

```

make_sig_table <- function(means_df, value_col, pvals_df, out_csv,
                           sorghum_name = "Sorghum",
                           wcc_name = "Sorghum + WCC") {

  wide <- means_df |>
    select(year, date, Treatment, !!sym(value_col)) |>
    tidyr::pivot_wider(names_from = Treatment, values_from = !!sym(value_col)) |>
    rename(
      sorghum = !!sorghum_name,
      wcc      = !!wcc_name
    ) |>
    mutate(
      diff_sorghum_minus_wcc = sorghum - wcc
    ) |>
    filter(!is.na(sorghum) & !is.na(wcc))

  sig_tbl <- wide |>
    left_join(pvals_df, by = c("year", "date")) |>
    mutate(
      p = as.numeric(p),
      sig = !is.na(p) & p < 0.05,
      wcc_lower = wcc < sorghum,
      sig_wcc_lower = sig & wcc_lower
    )

  readr::write_csv(sig_tbl, out_csv)
  sig_tbl
}

# Temperature table (uses moist_temp means + pvals_temp from your t-tests)
temp_sig_table <- make_sig_table(
  means_df = moist_temp,
  value_col = "mean_soil_temp",
  pvals_df = pvals_temp,
  out_csv = "data/processed/soil_temp_sig_by_date.csv"
)

# Moisture table
moist_sig_table <- make_sig_table(
  means_df = moist_temp,
  value_col = "mean_soil_moisture",
  pvals_df = pvals_moist,
  out_csv = "data/processed/soil_moist_sig_by_date.csv"
)

# ---- Summaries needed for manuscript text ----

summarize_sig <- function(sig_tbl) {
  # X out of Y where significant differences exist (either direction)
  Y <- sum(sig_tbl$sig, na.rm = TRUE)
  X <- sum(sig_tbl$sig_wcc_lower, na.rm = TRUE)

  by_year <- sig_tbl |>

```

```

group_by(year) |>
summarise(
  Y = sum(sig, na.rm = TRUE),
  X = sum(sig_wcc_lower, na.rm = TRUE),
  .groups = "drop"
)

# Range of sorghum minus wcc differences on significant-lower dates
diff_rng <- sig_tbl |>
filter(sig_wcc_lower) |>
summarise(
  min_diff = min(diff_sorghum_minus_wcc, na.rm = TRUE),
  max_diff = max(diff_sorghum_minus_wcc, na.rm = TRUE)
)

list(X = X, Y = Y, by_year = by_year, diff_rng = diff_rng)
}

temp_sum <- summarize_sig(temp_sig_table)
moist_sum <- summarize_sig(moist_sig_table)

# Print strings you can copy into manuscript drafts
cat(
  "Temperature: WCC significantly LOWER than Sorghum on ",
  temp_sum$X, " out of ", temp_sum$Y,
  " significant dates (",
  paste0(temp_sum$by_year$year, ": ", temp_sum$by_year$X, "/", temp_sum$by_year$Y, collapse = ", "),
  "). Sorghum was ",
  round(temp_sum$diff_rng$min_diff, 2), "-", round(temp_sum$diff_rng$max_diff, 2),
  " °C warmer on those dates (p < 0.05).\n",
  sep = ""
)

```

Temperature: WCC significantly LOWER than Sorghum on 13 out of 17 significant dates (2023: 6/9, 2024: 7/

```

cat(
  "Moisture: WCC significantly LOWER than Sorghum on ",
  moist_sum$X, " out of ", moist_sum$Y,
  " significant dates (",
  paste0(moist_sum$by_year$year, ": ", moist_sum$by_year$X, "/", moist_sum$by_year$Y, collapse = ", "),
  "). Sorghum was ",
  round(moist_sum$diff_rng$min_diff, 3), "-", round(moist_sum$diff_rng$max_diff, 3),
  " cm3 cm-3 higher on those dates (p < 0.05).\n",
  sep = ""
)

```

Moisture: WCC significantly LOWER than Sorghum on 9 out of 9 significant dates (2023: 9/9, 2024: 0/0). 3 higher on those dates (p < 0.05).

```
cat("Saved: data/processed/soil_temp_sig_by_date.csv\n")
```

Saved: data/processed/soil_temp_sig_by_date.csv

```
cat("Saved: data/processed/soil_moist_sig_by_date.csv\n")
```

Saved: data/processed/soil_moist_sig_by_date.csv

6 Figure 3: Sorghum and Sorghum + WCC biomass

7 Figure 4: Mean daily and cumulative nitrous oxide emissions

```
# Prepare Flux Data
flux_subset <- flux_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(
    Treatment = if_else(
      treatment == "Sorghum + Rye",
      "Sorghum + WCC",
      treatment
    ),
    year = year(date)
  ) |>
  filter(year %in% c(2023, 2024))

# 1. Daily Means and SE (for top panel)
daily_flux <- flux_subset |>
  group_by(Treatment, year, date) |>
  summarise(
    mean_flux = mean(gnha_day_no_negative, na.rm = TRUE) / 1000, # kg N ha-1 day-1
    se_flux = sd(gnha_day_no_negative, na.rm = TRUE) /
      sqrt(sum(!is.na(gnha_day_no_negative))) /
      1000,
    .groups = "drop"
  )

# 2. Significance (Wilcoxon on plot means)
daily_significance <- flux_subset |>
  group_by(year, date, Treatment, plot) |>
  summarise(
    plot_mean_flux = mean(gnha_day_no_negative, na.rm = TRUE),
    .groups = "drop"
  ) |>
  group_by(year, date) |>
  summarise(
    p_value = {
      sorghum_vals <- plot_mean_flux[Treatment == "Sorghum"]
      wcc_vals <- plot_mean_flux[Treatment == "Sorghum + WCC"]
      sorghum_vals <- sorghum_vals[!is.na(sorghum_vals)]
      wcc_vals <- wcc_vals[!is.na(wcc_vals)]

      if (length(sorghum_vals) >= 3 && length(wcc_vals) >= 3) {
        tryCatch(
          wilcox.test(sorghum_vals, wcc_vals, exact = FALSE)$p.value,
          error = function(e) NA_real_
        )
      } else {
        NA_real_
      }
    },
    .groups = "drop"
  )
```

```

) |>
mutate(
  significant = p_value < 0.05,
  star_label = if_else(significant & !is.na(p_value), "*", "")
)

# Star positions
max_flux_by_date <- daily_flux |>
  group_by(year, date) |>
  summarise(max_flux = max(mean_flux + se_flux, na.rm = TRUE), .groups = "drop")

star_data <- daily_significance |>
  left_join(max_flux_by_date, by = c("year", "date")) |>
  filter(star_label == "*") |>
  mutate(star_y = max_flux * 1.1)

# 3. Cumulative Flux Calculation (CORRECTED)

# First: Aggregate to one value per plot per day (averaging Row/Interrow)
daily_flux_plot_level <- flux_subset |>
  group_by(Treatment, year, plot, date) |>
  summarise(
    daily_flux = mean(gnha_day_no_negative, na.rm = TRUE) / 1000, # kg N
    .groups = "drop"
  )

# Second: Interpolate and Accumulate
cum_flux_plot_level <- daily_flux_plot_level |>
  arrange(Treatment, year, plot, date) |>
  group_by(Treatment, year, plot) |>
  group_modify(~ {
    # Complete the date sequence for this specific plot
    df <- .x |>
      complete(date = seq(min(date), max(date), by = "day")) |>
      arrange(date)

    non_na <- !is.na(df$daily_flux)

    # Skip if insufficient data
    if (sum(non_na) == 0) return(tibble())

    # Interpolate
    flux_interp <- if (sum(non_na) >= 2) {
      approx(
        x = df$date[non_na],
        y = df$daily_flux[non_na],
        xout = df$date,
        rule = 1, # NA outside range
        ties = "ordered"
      )$y
    } else {
      # Fallback for single point (rare)
      rep(df$daily_flux[non_na][1], nrow(df))
    }
  })

```

```

}

# Accumulate and filter out NAs (e.g. before first sampling)
df |>
  mutate(
    flux_interp = flux_interp,
    cum_flux = cumsum(flux_interp)
  ) |>
  filter(!is.na(flux_interp))
}) |>
ungroup()

# Third: Summary Statistics for Plotting
cum_flux_summary <- cum_flux_plot_level |>
  group_by(Treatment, year, date) |>
  summarise(
    cum_flux_mean = mean(cum_flux, na.rm = TRUE),
    cum_flux_sd = sd(cum_flux, na.rm = TRUE),
    n_plots = n(),
    cum_flux_se = cum_flux_sd / sqrt(n_plots),
    t_crit = if_else(n_plots > 1, qt(0.975, df = n_plots - 1), NA_real_),
    cum_flux_ci = t_crit * cum_flux_se,
    .groups = "drop"
  )

# Precipitation
flux_date_ranges <- daily_flux |>
  group_by(year) |>
  summarise(min_date = min(date), max_date = max(date), .groups = "drop")
daily_precip_trimmed <- rain |>
  select(date, year, precipmm) |>
  filter(year %in% c(2023, 2024)) |>
  left_join(flux_date_ranges, by = "year") |>
  filter(date >= min_date, date <= max_date)
fert_events <- tibble(
  year = c(2023, 2024),
  fertilizer_date = as.Date(c("2023-05-05", "2024-07-17"))
)

# Plotting Constants
global_max_flux <- max(daily_flux$mean_flux + daily_flux$se_flux, na.rm = TRUE)
global_max_precip <- max(daily_precip_trimmed$precipmm, na.rm = TRUE)
global_max_cum_flux <- max(
  cum_flux_summary$cum_flux_mean + cum_flux_summary$cum_flux_ci,
  na.rm = TRUE
)
global_precip_scale_factor <- global_max_flux * 0.4 / global_max_precip
global_plot_top <- global_max_flux * 1.1

# Plot Function
create_dual_axis_plot <- function(year_val, is_right_panel = FALSE) {
  flux_year <- filter(daily_flux, year == year_val)
  precip_year <- filter(daily_precip_trimmed, year == year_val)

```

```

star_year <- filter(star_data, year == year_val)
fert_year <- filter(fert_events, year == year_val)
cum_flux_year <- filter(cum_flux_summary, year == year_val)
year_start <- as.Date(paste0(year_val, "-01-01"))
year_end <- as.Date(paste0(year_val, "-12-31"))

daily_plot <- ggplot() +
  geom_rect(
    data = precip_year,
    aes(
      xmin = date - 0.5,
      xmax = date + 0.5,
      ymin = global_plot_top - precipmm * global_precip_scale_factor,
      ymax = global_plot_top
    ),
    fill = "#377EB8",
    alpha = 0.7
  ) +
  geom_point(
    data = flux_year,
    aes(x = date, y = mean_flux, color = Treatment),
    size = 1.5
  ) +
  geom_errorbar(
    data = flux_year,
    aes(
      x = date,
      ymin = mean_flux - se_flux,
      ymax = mean_flux + se_flux,
      color = Treatment
    ),
    width = 0.2,
    alpha = 0.5
  ) +
  geom_vline(
    data = fert_year,
    aes(xintercept = fertilizer_date),
    linetype = "dashed",
    color = "black",
    alpha = 0.7
  ) +
  geom_text(
    data = star_year,
    aes(x = date, y = star_y, label = star_label),
    color = "black",
    size = 5
  ) +
  guides(color = "none") +
  scale_color_manual(values = treatment_colors) +
  scale_x_date(
    date_breaks = "2 months",
    date_labels = "%b",
    limits = c(year_start, year_end)
  )

```

```

) +
scale_y_continuous(
  name = if (!is_right_panel) {
    expression("Daily N"[2] * "0 Flux (kg N ha"-1 * " d"-1 * ")")
  } else {
    ""
  },
  limits = c(0, global_plot_top),
  sec.axis = sec_axis(
    ~ (global_plot_top - .) / global_precip_scale_factor,
    name = if (is_right_panel) "Daily Precipitation (mm)" else ""
  )
) +
theme_publication(base_size = 10) +
theme(
  legend.position = "none",
  axis.text.x = element_blank(),
  axis.ticks.x = element_blank(),
  axis.line.x = element_blank(),
  axis.text.y = if (is_right_panel) element_blank() else element_text(),
  axis.ticks.y = if (is_right_panel) element_blank() else element_line(),
  axis.line.y = if (is_right_panel) element_blank() else element_line(),
  axis.title.y = if (is_right_panel) element_blank() else element_text(),
  axis.text.y.right = if (is_right_panel) {
    element_text(color = "#377EB8")
  } else {
    element_blank()
  },
  axis.ticks.y.right = if (is_right_panel) {
    element_line()
  } else {
    element_blank()
  },
  axis.title.y.right = if (is_right_panel) {
    element_text(color = "#377EB8")
  } else {
    element_blank()
  }
) +
labs(x = "") +
geom_hline(yintercept = 0, linetype = "solid", alpha = 0.3)

cumulative_plot <- ggplot(
  cum_flux_year,
  aes(x = date, y = cum_flux_mean, color = Treatment, fill = Treatment)
) +
geom_ribbon(
  aes(
    ymin = pmax(cum_flux_mean - cum_flux_ci, 0),
    ymax = cum_flux_mean + cum_flux_ci
  ),
  alpha = 0.15,
  linewidth = 0

```

```

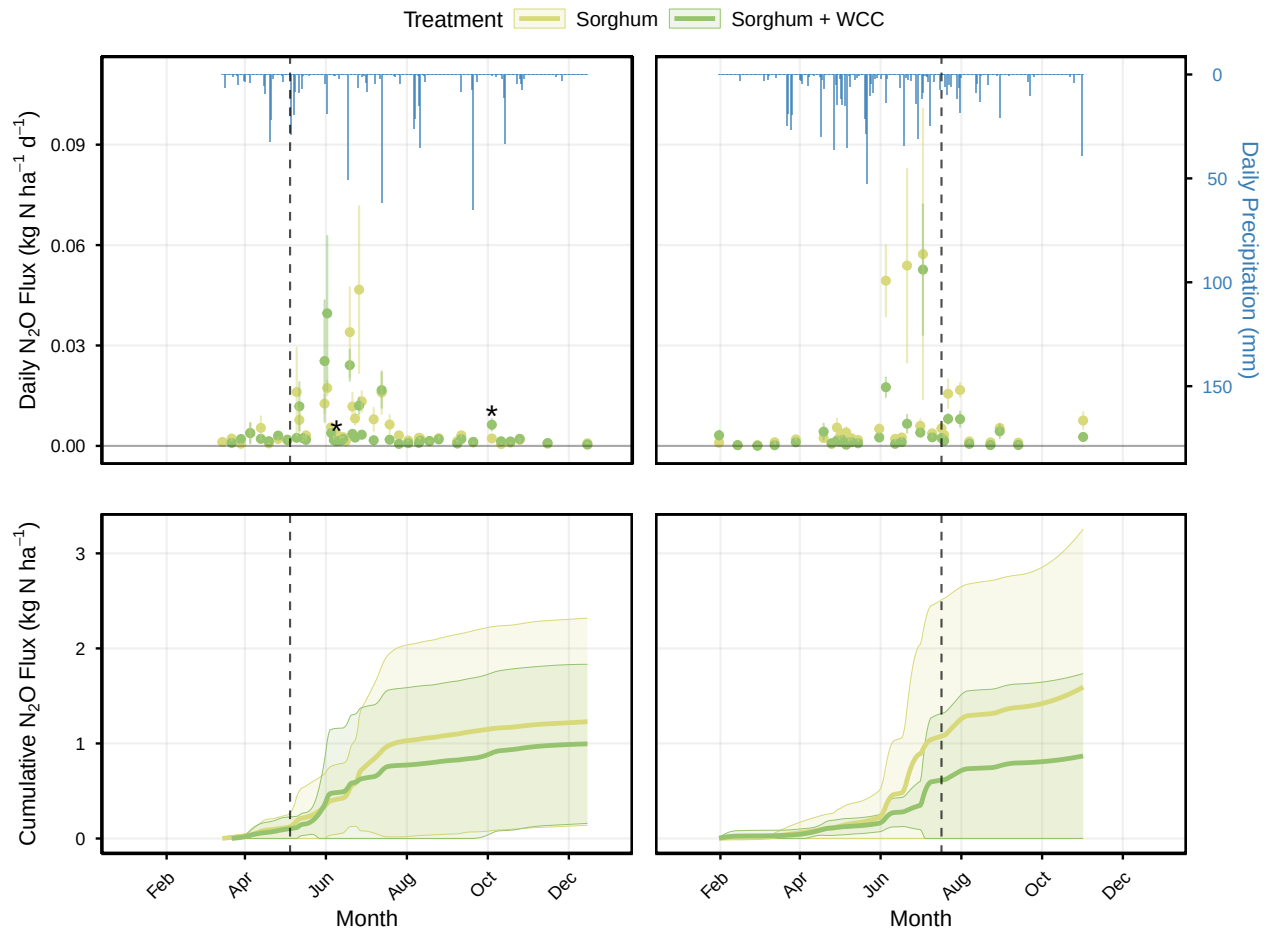
) +
geom_line(linewidth = 1) +
geom_vline(
  data = fert_year,
  aes(xintercept = fertilizer_date),
  linetype = "dashed",
  color = "black",
  alpha = 0.7
) +
scale_color_manual(values = treatment_colors) +
scale_fill_manual(values = treatment_colors) +
scale_x_date(
  date_breaks = "2 months",
  date_labels = "%b",
  limits = c(year_start, year_end)
) +
scale_y_continuous(
  name = if (!is_right_panel) {
    expression("Cumulative N"[2] * "0 Flux (kg N ha"-1 * ")")
  } else {
    ""
  },
  limits = c(0, global_max_cum_flux)
) +
labs(x = "Month") +
theme_publication(base_size = 10) +
theme(
  axis.text.y = if (is_right_panel) element_blank() else element_text(),
  axis.ticks.y = if (is_right_panel) element_blank() else element_line(),
  axis.line.y = if (is_right_panel) element_blank() else element_line(),
  axis.title.y = if (is_right_panel) element_blank() else element_text()
)

(daily_plot / cumulative_plot) + patchwork::plot_layout(heights = c(1.2, 1))
}

figure3_dual_axis <- (create_dual_axis_plot(2023) |
  create_dual_axis_plot(2024, TRUE)) +
  patchwork::plot_layout(guides = "collect") &
  theme(legend.position = "top", legend.justification = "center")

figure3_dual_axis

```



```
ggsave(
  filename = "figures/figure2_flux_precip_cumulative_2023_2024_plotlevelCI_dualaxis.png",
  plot = figure3_dual_axis,
  width = 174,
  height = 174,
  units = "mm",
  dpi = 600,
  bg = "white"
)
```

8 plot-level cumulative range accross all sorghum sorghum + WCC and corn for the 2 years

```
# Filter data for Sorghum, Sorghum + WCC, and Corn across 2023 and 2024
flux_range_data <- flux_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye", "Corn")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    year = year(date)
  ) |>
  filter(year %in% c(2023, 2024))

# Calculate daily flux per plot (averaging row/interrow if present)
```

```

daily_flux_plot <- flux_range_data |>
  group_by(Treatment, year, plot, date) |>
  summarise(
    daily_flux = mean(gnha_day_no_negative, na.rm = TRUE) / 1000, # kg N ha-1
    .groups = "drop"
  )

# Extract the Range in GRAMS
range_stats <- daily_flux_plot |>
  summarise(
    min_flux_g = min(daily_flux),
    max_flux_g = max(daily_flux)
  )

# 3. Check specific High Values (to verify the ">600" claim)
top_fluxes <- daily_flux_plot |>
  arrange(desc(daily_flux)) |>
  slice(1:20) |>
  select(year, Treatment, date, daily_flux)

# Print results
print(range_stats)

```

```
# A tibble: 1 x 2
```

```

  min_flux_g max_flux_g
    <dbl>      <dbl>
1         0         0.419

```

```
print("Top 5 Flux Events (g N ha-1 d-1):")
```

```
[1] "Top 5 Flux Events (g N ha-1 d-1):"
```

```
print(top_fluxes)
```

```
# A tibble: 20 x 4
```

```

  year Treatment      date      daily_flux
  <dbl> <chr>          <date>         <dbl>
1  2024 Corn          2024-05-03      0.419
2  2023 Corn          2023-06-19      0.204
3  2024 Corn          2024-06-21      0.180
4  2024 Corn          2024-05-10      0.153
5  2024 Sorghum + WCC 2024-07-03      0.138
6  2023 Sorghum       2023-06-26      0.127
7  2024 Sorghum       2024-06-21      0.124
8  2023 Corn          2023-06-26      0.122
9  2023 Corn          2023-06-19      0.111
10 2024 Sorghum       2024-07-03      0.106
11 2023 Sorghum + WCC 2023-06-02      0.103
12 2023 Corn          2023-07-13      0.101
13 2023 Corn          2023-06-21      0.0913
14 2023 Sorghum       2023-06-19      0.0858
15 2024 Corn          2024-05-08      0.0803

```

16	2023 Sorghum + WCC	2023-05-31	0.0788
17	2024 Corn	2024-06-05	0.0776
18	2024 Sorghum	2024-06-21	0.0762
19	2023 Corn	2023-06-28	0.0729
20	2024 Sorghum	2024-06-05	0.0666

```
# Interpolate daily values and calculate cumulative sum for each plot
```

```
cum_flux_plot <- daily_flux_plot |>
  arrange(Treatment, year, plot, date) |>
  group_by(Treatment, year, plot) |>
  group_modify(~ {
    # Create full date sequence for the plot's observation period
    df <- .x |>
      complete(date = seq(min(date), max(date), by = "day")) |>
      arrange(date)

    non_na <- !is.na(df$daily_flux)
    if (sum(non_na) == 0) return(tibble())

    # Linear interpolation
    flux_interp <- if (sum(non_na) >= 2) {
      approx(
        x = df$date[non_na],
        y = df$daily_flux[non_na],
        xout = df$date,
        rule = 1, # NA outside observed range
        ties = "ordered"
      )$y
    } else {
      rep(df$daily_flux[non_na][1], nrow(df))
    }

    df |>
      mutate(
        flux_interp = flux_interp,
        cum_flux = cumsum(replace_na(flux_interp, 0))
      ) |>
      filter(!is.na(flux_interp))
  }) |>
  ungroup()

# Extract the final cumulative total for each plot
final_cum_values <- cum_flux_plot |>
  group_by(Treatment, year, plot) |>
  summarise(total_cum = max(cum_flux), .groups = "drop")

# Calculate and print the range across all plots
overall_range <- range(final_cum_values$total_cum)
cat("Overall cumulative N2O flux range (kg N ha-1) across Sorghum, Sorghum + WCC, and Corn (2023-2024):",
    paste(round(overall_range, 2), collapse = " - "), "\n")
```

Overall cumulative N2O flux range (kg N ha⁻¹) across Sorghum, Sorghum + WCC, and Corn (2023-2024): 0.38 - 3.79

9 Daily flux details

```
library(gt)
str(flux_data)

tibble [2,149 x 11] (S3: tbl_df/tbl/data.frame)
 $ date          : Date [1:2149], format: "2023-03-15" "2023-03-15" ...
 $ plot          : chr [1:2149] "01" "01" "02" "02" ...
 $ rowvs_interrow : chr [1:2149] "Interrow" "Row" "Interrow" "Row" ...
 $ best_flux_nmol_1m_2s_1: num [1:2149] 0.2223 0.11477 -0.02153 0.01361 -0.00862 ...
 $ ts_2_mean     : num [1:2149] 13 11.5 11.2 11.2 10.8 ...
 $ swc_2_mean    : num [1:2149] 0.422 0.421 0.498 0.497 0.315 ...
 $ gnha_day      : num [1:2149] 5.38 2.778 -0.521 0.329 -0.209 ...
 $ gnha_day_no_negative : num [1:2149] 5.38 2.778 0 0.329 0 ...
 $ row_id        : num [1:2149] 1 2 3 4 5 6 7 8 9 10 ...
 $ year          : num [1:2149] 2023 2023 2023 2023 2023 ...
 $ treatment     : chr [1:2149] "Corn" "Corn" "Sorghum" "Sorghum" ...

# Table S1: Use RAW flux values (including negatives) for min; round to whole numbers
# If 'gnha_day' column exists use it for min; otherwise fall back to gnha_day_no_negative
raw_flux_col <- if ("gnha_day" %in% names(flux_data)) "gnha_day" else "gnha_day_no_negative"

daily_flux_stats <- flux_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye", "Corn", "Soy")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    year = year(date),
    raw_flux = .data[[raw_flux_col]]
  ) |>
  filter(year %in% c(2023, 2024)) |>

# --- STEP 1: POOL ROW & INTERROW ---
group_by(date, plot, Treatment, year) |>
summarise(
  plot_daily_flux = mean(gnha_day_no_negative, na.rm = TRUE),
  plot_daily_flux_raw = mean(raw_flux, na.rm = TRUE),
  .groups = "drop"
) |>

# --- STEP 2: CALCULATE STATS - use raw for min to capture negatives ---
group_by(Treatment, year) |>
summarise(
  mean_flux = mean(plot_daily_flux, na.rm = TRUE),
  median_flux = median(plot_daily_flux, na.rm = TRUE),
  min_flux = min(plot_daily_flux_raw, na.rm = TRUE), # negative values preserved
  max_flux = max(plot_daily_flux, na.rm = TRUE),
  .groups = "drop"
)
daily_flux_stats |>
  group_by(Treatment) |>
  summarise(mean = mean(mean_flux))

# A tibble: 4 x 2
  Treatment      mean
  <chr>          <dbl>
```

```

1 Corn          14.1
2 Sorghum       6.91
3 Sorghum + WCC 4.63
4 Soy           5.68

```

```
print(daily_flux_stats)
```

```
# A tibble: 8 x 6
```

	Treatment	year	mean_flux	median_flux	min_flux	max_flux
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Corn	2023	11.4	3.01	-2.73	204.
2	Corn	2024	16.8	3.32	-1.33	419.
3	Sorghum	2023	6.37	2.11	-0.790	127.
4	Sorghum	2024	7.45	2.15	-5.45	124.
5	Sorghum + WCC	2023	4.92	1.79	0.0350	103.
6	Sorghum + WCC	2024	4.34	1.22	-20.7	138.
7	Soy	2023	6.00	2.55	-0.489	101.
8	Soy	2024	5.36	1.36	-17.2	130.

```
# Create formatted table - round to nearest whole number per McDaniel
```

```
supp_table1 <- daily_flux_stats %>%
```

```
  gt() %>%
```

```
  cols_label(
```

```
    Treatment = "Treatment",
```

```
    year = "Year",
```

```
    mean_flux = "Mean",
```

```
    median_flux = "Median",
```

```
    min_flux = "Min",
```

```
    max_flux = "Max"
```

```
  ) %>%
```

```
  fmt_number(
```

```
    columns = c(mean_flux, median_flux, min_flux, max_flux),
```

```
    decimals = 0 # Round to nearest whole number
```

```
  ) %>%
```

```
  tab_header(
```

```
    title = md("**Table S1. Daily NO Flux Statistics**"),
```

```
    subtitle = "Values represent plot-level averages (pooled row/interrow) in g NO-N ha-1 d-1. Min value
```

```
  ) %>%
```

```
  cols_align(aligned = "center", columns = everything()) %>%
```

```
  cols_align(aligned = "left", columns = "Treatment")
```

```
print(supp_table1)
```

```
<div id="ohtwitnzlj" style="padding-left:0px;padding-right:0px;padding-top:10px;padding-
bottom:10px;overflow-x:auto;overflow-y:auto;width:auto;height:auto;">
```

```
<style>#ohtwitnzlj table {
```

```
  font-family: system-ui, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif, 'Apple Color Emoji', 'Segoe
```

```
  -webkit-font-smoothing: antialiased;
```

```
  -moz-osx-font-smoothing: grayscale;
```

```
}
```

```
#ohtwitnzlj thead, #ohtwitnzlj tbody, #ohtwitnzlj tfoot, #ohtwitnzlj tr, #ohtwitnzlj td, #ohtwitnzlj th
```

```
  border-style: none;
```

```
}
```

```

#ohwitnznlj p {
  margin: 0;
  padding: 0;
}

#ohwitnznlj .gt_table {
  display: table;
  border-collapse: collapse;
  line-height: normal;
  margin-left: auto;
  margin-right: auto;
  color: #333333;
  font-size: 16px;
  font-weight: normal;
  font-style: normal;
  background-color: #FFFFFF;
  width: auto;
  border-top-style: solid;
  border-top-width: 2px;
  border-top-color: #A8A8A8;
  border-right-style: none;
  border-right-width: 2px;
  border-right-color: #D3D3D3;
  border-bottom-style: solid;
  border-bottom-width: 2px;
  border-bottom-color: #A8A8A8;
  border-left-style: none;
  border-left-width: 2px;
  border-left-color: #D3D3D3;
}

#ohwitnznlj .gt_caption {
  padding-top: 4px;
  padding-bottom: 4px;
}

#ohwitnznlj .gt_title {
  color: #333333;
  font-size: 125%;
  font-weight: initial;
  padding-top: 4px;
  padding-bottom: 4px;
  padding-left: 5px;
  padding-right: 5px;
  border-bottom-color: #FFFFFF;
  border-bottom-width: 0;
}

#ohwitnznlj .gt_subtitle {
  color: #333333;
  font-size: 85%;
  font-weight: initial;
  padding-top: 3px;
  padding-bottom: 5px;
}

```

```

padding-left: 5px;
padding-right: 5px;
border-top-color: #FFFFFF;
border-top-width: 0;
}

#ohtwitnztlj .gt_heading {
background-color: #FFFFFF;
text-align: center;
border-bottom-color: #FFFFFF;
border-left-style: none;
border-left-width: 1px;
border-left-color: #D3D3D3;
border-right-style: none;
border-right-width: 1px;
border-right-color: #D3D3D3;
}

#ohtwitnztlj .gt_bottom_border {
border-bottom-style: solid;
border-bottom-width: 2px;
border-bottom-color: #D3D3D3;
}

#ohtwitnztlj .gt_col_headings {
border-top-style: solid;
border-top-width: 2px;
border-top-color: #D3D3D3;
border-bottom-style: solid;
border-bottom-width: 2px;
border-bottom-color: #D3D3D3;
border-left-style: none;
border-left-width: 1px;
border-left-color: #D3D3D3;
border-right-style: none;
border-right-width: 1px;
border-right-color: #D3D3D3;
}

#ohtwitnztlj .gt_col_heading {
color: #333333;
background-color: #FFFFFF;
font-size: 100%;
font-weight: normal;
text-transform: inherit;
border-left-style: none;
border-left-width: 1px;
border-left-color: #D3D3D3;
border-right-style: none;
border-right-width: 1px;
border-right-color: #D3D3D3;
vertical-align: bottom;
padding-top: 5px;
padding-bottom: 6px;
}

```

```

padding-left: 5px;
padding-right: 5px;
overflow-x: hidden;
}

#ohtwitnztj .gt_column_spanner_outer {
color: #333333;
background-color: #FFFFFF;
font-size: 100%;
font-weight: normal;
text-transform: inherit;
padding-top: 0;
padding-bottom: 0;
padding-left: 4px;
padding-right: 4px;
}

#ohtwitnztj .gt_column_spanner_outer:first-child {
padding-left: 0;
}

#ohtwitnztj .gt_column_spanner_outer:last-child {
padding-right: 0;
}

#ohtwitnztj .gt_column_spanner {
border-bottom-style: solid;
border-bottom-width: 2px;
border-bottom-color: #D3D3D3;
vertical-align: bottom;
padding-top: 5px;
padding-bottom: 5px;
overflow-x: hidden;
display: inline-block;
width: 100%;
}

#ohtwitnztj .gt_spanner_row {
border-bottom-style: hidden;
}

#ohtwitnztj .gt_group_heading {
padding-top: 8px;
padding-bottom: 8px;
padding-left: 5px;
padding-right: 5px;
color: #333333;
background-color: #FFFFFF;
font-size: 100%;
font-weight: initial;
text-transform: inherit;
border-top-style: solid;
border-top-width: 2px;
border-top-color: #D3D3D3;

```

```

border-bottom-style: solid;
border-bottom-width: 2px;
border-bottom-color: #D3D3D3;
border-left-style: none;
border-left-width: 1px;
border-left-color: #D3D3D3;
border-right-style: none;
border-right-width: 1px;
border-right-color: #D3D3D3;
vertical-align: middle;
text-align: left;
}

#ohtwitnztlj .gt_empty_group_heading {
padding: 0.5px;
color: #333333;
background-color: #FFFFFF;
font-size: 100%;
font-weight: initial;
border-top-style: solid;
border-top-width: 2px;
border-top-color: #D3D3D3;
border-bottom-style: solid;
border-bottom-width: 2px;
border-bottom-color: #D3D3D3;
vertical-align: middle;
}

#ohtwitnztlj .gt_from_md > :first-child {
margin-top: 0;
}

#ohtwitnztlj .gt_from_md > :last-child {
margin-bottom: 0;
}

#ohtwitnztlj .gt_row {
padding-top: 8px;
padding-bottom: 8px;
padding-left: 5px;
padding-right: 5px;
margin: 10px;
border-top-style: solid;
border-top-width: 1px;
border-top-color: #D3D3D3;
border-left-style: none;
border-left-width: 1px;
border-left-color: #D3D3D3;
border-right-style: none;
border-right-width: 1px;
border-right-color: #D3D3D3;
vertical-align: middle;
overflow-x: hidden;
}

```

```

#ohtwitnzlj .gt_stub {
  color: #333333;
  background-color: #FFFFFF;
  font-size: 100%;
  font-weight: initial;
  text-transform: inherit;
  border-right-style: solid;
  border-right-width: 2px;
  border-right-color: #D3D3D3;
  padding-left: 5px;
  padding-right: 5px;
}

#ohtwitnzlj .gt_stub_row_group {
  color: #333333;
  background-color: #FFFFFF;
  font-size: 100%;
  font-weight: initial;
  text-transform: inherit;
  border-right-style: solid;
  border-right-width: 2px;
  border-right-color: #D3D3D3;
  padding-left: 5px;
  padding-right: 5px;
  vertical-align: top;
}

#ohtwitnzlj .gt_row_group_first td {
  border-top-width: 2px;
}

#ohtwitnzlj .gt_row_group_first th {
  border-top-width: 2px;
}

#ohtwitnzlj .gt_summary_row {
  color: #333333;
  background-color: #FFFFFF;
  text-transform: inherit;
  padding-top: 8px;
  padding-bottom: 8px;
  padding-left: 5px;
  padding-right: 5px;
}

#ohtwitnzlj .gt_first_summary_row {
  border-top-style: solid;
  border-top-color: #D3D3D3;
}

#ohtwitnzlj .gt_first_summary_row.thick {
  border-top-width: 2px;
}

```

```

#ohtwitnztj .gt_last_summary_row {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
}

#ohtwitnztj .gt_grand_summary_row {
    color: #333333;
    background-color: #FFFFFF;
    text-transform: inherit;
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
}

#ohtwitnztj .gt_first_grand_summary_row {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    border-top-style: double;
    border-top-width: 6px;
    border-top-color: #D3D3D3;
}

#ohtwitnztj .gt_last_grand_summary_row_top {
    padding-top: 8px;
    padding-bottom: 8px;
    padding-left: 5px;
    padding-right: 5px;
    border-bottom-style: double;
    border-bottom-width: 6px;
    border-bottom-color: #D3D3D3;
}

#ohtwitnztj .gt_stripped {
    background-color: rgba(128, 128, 128, 0.05);
}

#ohtwitnztj .gt_table_body {
    border-top-style: solid;
    border-top-width: 2px;
    border-top-color: #D3D3D3;
    border-bottom-style: solid;
    border-bottom-width: 2px;
    border-bottom-color: #D3D3D3;
}

```

```
#ohtwitnztj .gt_footnotes {
  color: #333333;
  background-color: #FFFFFF;
  border-bottom-style: none;
  border-bottom-width: 2px;
  border-bottom-color: #D3D3D3;
  border-left-style: none;
  border-left-width: 2px;
  border-left-color: #D3D3D3;
  border-right-style: none;
  border-right-width: 2px;
  border-right-color: #D3D3D3;
}
```

```
#ohtwitnztj .gt_footnote {
  margin: 0px;
  font-size: 90%;
  padding-top: 4px;
  padding-bottom: 4px;
  padding-left: 5px;
  padding-right: 5px;
}
```

```
#ohtwitnztj .gt_sourcenotes {
  color: #333333;
  background-color: #FFFFFF;
  border-bottom-style: none;
  border-bottom-width: 2px;
  border-bottom-color: #D3D3D3;
  border-left-style: none;
  border-left-width: 2px;
  border-left-color: #D3D3D3;
  border-right-style: none;
  border-right-width: 2px;
  border-right-color: #D3D3D3;
}
```

```
#ohtwitnztj .gt_sourcenote {
  font-size: 90%;
  padding-top: 4px;
  padding-bottom: 4px;
  padding-left: 5px;
  padding-right: 5px;
}
```

```
#ohtwitnztj .gt_left {
  text-align: left;
}
```

```
#ohtwitnztj .gt_center {
  text-align: center;
}
```

```
#ohtwitnztj .gt_right {
```

```

    text-align: right;
    font-variant-numeric: tabular-nums;
}

#ohtwitnztj .gt_font_normal {
    font-weight: normal;
}

#ohtwitnztj .gt_font_bold {
    font-weight: bold;
}

#ohtwitnztj .gt_font_italic {
    font-style: italic;
}

#ohtwitnztj .gt_super {
    font-size: 65%;
}

#ohtwitnztj .gt_footnote_marks {
    font-size: 75%;
    vertical-align: 0.4em;
    position: initial;
}

#ohtwitnztj .gt_asterisk {
    font-size: 100%;
    vertical-align: 0;
}

#ohtwitnztj .gt_indent_1 {
    text-indent: 5px;
}

#ohtwitnztj .gt_indent_2 {
    text-indent: 10px;
}

#ohtwitnztj .gt_indent_3 {
    text-indent: 15px;
}

#ohtwitnztj .gt_indent_4 {
    text-indent: 20px;
}

#ohtwitnztj .gt_indent_5 {
    text-indent: 25px;
}

#ohtwitnztj .katex-display {
    display: inline-flex !important;
    margin-bottom: 0.75em !important;
}

```

```

}

#ohtwitnzlj div.Reactable > div.rt-table > div.rt-thead > div.rt-tr.rt-tr-group-header > div.rt-
th-group:after {
  height: 0px !important;
}
</style>
<table class="gt_table" data-quarto-disable-processing="false" data-quarto-bootstrap="false">
  <thead>
    <tr class="gt_heading">
      <td colspan="6" class="gt_heading gt_title gt_font_normal" style><span data-qmd-
base64="KipUYWJsZSBTMS4gRGFpbHkgTuKCgk8gRmx1eCBTdGF0aXN0aWNzKio="><span class='gt_from_md'><strong>Tabl
</tr>
    <tr class="gt_heading">
      <td colspan="6" class="gt_heading gt_subtitle gt_font_normal gt_bottom_border" style>Values repre
level averages (pooled row/interrow) in g NO-N ha-1 d-1. Min values include negative (uptake) fluxes.</t
</tr>
    <tr class="gt_col_headings">
      <th class="gt_col_heading gt_columns_bottom_border gt_left" rowspan="1" colspan="1" scope="col" i
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" scope="col"
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" scope="col"
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" scope="col"
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" scope="col"
      <th class="gt_col_heading gt_columns_bottom_border gt_center" rowspan="1" colspan="1" scope="col"
    </tr>
  </thead>
  <tbody class="gt_table_body">
    <tr><td headers="Treatment" class="gt_row gt_left">Corn</td>
<td headers="year" class="gt_row gt_center">2023</td>
<td headers="mean_flux" class="gt_row gt_center">11</td>
<td headers="median_flux" class="gt_row gt_center">3</td>
<td headers="min_flux" class="gt_row gt_center">-3</td>
<td headers="max_flux" class="gt_row gt_center">204</td></tr>
    <tr><td headers="Treatment" class="gt_row gt_left">Corn</td>
<td headers="year" class="gt_row gt_center">2024</td>
<td headers="mean_flux" class="gt_row gt_center">17</td>
<td headers="median_flux" class="gt_row gt_center">3</td>
<td headers="min_flux" class="gt_row gt_center">-1</td>
<td headers="max_flux" class="gt_row gt_center">419</td></tr>
    <tr><td headers="Treatment" class="gt_row gt_left">Sorghum</td>
<td headers="year" class="gt_row gt_center">2023</td>
<td headers="mean_flux" class="gt_row gt_center">6</td>
<td headers="median_flux" class="gt_row gt_center">2</td>
<td headers="min_flux" class="gt_row gt_center">-1</td>
<td headers="max_flux" class="gt_row gt_center">127</td></tr>
    <tr><td headers="Treatment" class="gt_row gt_left">Sorghum</td>
<td headers="year" class="gt_row gt_center">2024</td>
<td headers="mean_flux" class="gt_row gt_center">7</td>
<td headers="median_flux" class="gt_row gt_center">2</td>
<td headers="min_flux" class="gt_row gt_center">-5</td>
<td headers="max_flux" class="gt_row gt_center">124</td></tr>
    <tr><td headers="Treatment" class="gt_row gt_left">Sorghum + WCC</td>
<td headers="year" class="gt_row gt_center">2023</td>
<td headers="mean_flux" class="gt_row gt_center">5</td>

```

```

<td headers="median_flux" class="gt_row gt_center">2</td>
<td headers="min_flux" class="gt_row gt_center">0</td>
<td headers="max_flux" class="gt_row gt_center">103</td></tr>
  <tr><td headers="Treatment" class="gt_row gt_left">Sorghum + WCC</td>
<td headers="year" class="gt_row gt_center">2024</td>
<td headers="mean_flux" class="gt_row gt_center">4</td>
<td headers="median_flux" class="gt_row gt_center">1</td>
<td headers="min_flux" class="gt_row gt_center">-21</td>
<td headers="max_flux" class="gt_row gt_center">138</td></tr>
  <tr><td headers="Treatment" class="gt_row gt_left">Soy</td>
<td headers="year" class="gt_row gt_center">2023</td>
<td headers="mean_flux" class="gt_row gt_center">6</td>
<td headers="median_flux" class="gt_row gt_center">3</td>
<td headers="min_flux" class="gt_row gt_center">0</td>
<td headers="max_flux" class="gt_row gt_center">101</td></tr>
  <tr><td headers="Treatment" class="gt_row gt_left">Soy</td>
<td headers="year" class="gt_row gt_center">2024</td>
<td headers="mean_flux" class="gt_row gt_center">5</td>
<td headers="median_flux" class="gt_row gt_center">1</td>
<td headers="min_flux" class="gt_row gt_center">-17</td>
<td headers="max_flux" class="gt_row gt_center">130</td></tr>
</tbody>

</table>
</div>

```

```
# save
```

```

library(dplyr)
library(gt)

# 1. Use your existing 'cum_flux_summary' (which already has Means & CIs)
# Filter for just the relevant treatments (No Corn) and the final date of each year
validation_stats <- cum_flux_summary |>
  filter(Treatment %in% c("Sorghum", "Sorghum + WCC")) |>
  group_by(year, Treatment) |>
  slice_max(date, n = 1) |> # Get the end-of-season cumulative total
  ungroup() |>
  select(year, Treatment, cum_flux_mean, cum_flux_se, cum_flux_ci)

# 2. Create the Publication-Ready Table
validation_table <- validation_stats |>
  mutate(
    # Create the "Mean (SE)" string often used in papers
    Mean_SE = sprintf("%.2f (%.2f)", cum_flux_mean, cum_flux_se),
    # Calculate 95% CI Range
    CI_Lower = cum_flux_mean - cum_flux_ci,
    CI_Upper = cum_flux_mean + cum_flux_ci,
    # Create a visual check for the "0.9 to 1.6" range statement
    In_Range_Check = case_when(
      cum_flux_mean >= 0.9 & cum_flux_mean <= 1.6 ~ " Matches Text",
      TRUE ~ " Check Value"
    )
  ) |>
  select(year, Treatment, Mean_SE, CI_Lower, CI_Upper, In_Range_Check) |>

```

Validation of Cumulative N O Statement

Verifying: 'Reduced non-significantly (Range: 0.9 to 1.6 kg N)'

year	Treatment	Mean Cumulative Flux (SE)	95% CI Lower	95% CI Upper	Text Validation
2023	Sorghum	1.23 (0.34)	0.14	2.32	Matches Text
2023	Sorghum + WCC	1.00 (0.26)	0.16	1.83	Matches Text
2024	Sorghum	1.59 (0.52)	-0.07	3.26	Matches Text
2024	Sorghum + WCC	0.87 (0.27)	0.00	1.73	Check Value

```
gt() |>
tab_header(
  title = md("**Validation of Cumulative N O Statement**"),
  subtitle = "Verifying: 'Reduced non-significantly (Range: 0.9 to 1.6 kg N)'"
) |>
cols_label(
  Mean_SE = "Mean Cumulative Flux (SE)",
  CI_Lower = "95% CI Lower",
  CI_Upper = "95% CI Upper",
  In_Range_Check = "Text Validation"
) |>
fmt_number(
  columns = c(CI_Lower, CI_Upper),
  decimals = 2
) |>
# Color code the rows to make it easy to spot the treatments
tab_style(
  style = list(cell_fill(color = "#f9f9f9")),
  locations = cells_body(rows = year == 2024)
)

# 3. Display and Save
validation_table
```

```
# Save it so you have a permanent record for the response letter
gtsave(validation_table, "figures/validation_table_cumulative_n2o.png")
```

10 Row vs. Interrow flux comparisons

```
glimpse(flux_data)
```

```
Rows: 2,149
Columns: 11
$ date          <date> 2023-03-15, 2023-03-15, 2023-03-15, 2023-03-15~
$ plot          <chr> "01", "01", "02", "02", "03", "03", "04", "05",~
$ rowvs_interrow <chr> "Interrow", "Row", "Interrow", "Row", "Interrow~
$ best_flux_nmol_1m_2s_1 <dbl> 0.22230, 0.11477, -0.02153, 0.01361, -0.00862, ~
$ ts_2_mean     <dbl> 13.03, 11.46, 11.25, 11.15, 10.81, 10.73, 9.45,~
$ swc_2_mean    <dbl> 0.422, 0.421, 0.498, 0.497, 0.315, 0.396, 0.436~
$ gnha_day      <dbl> 5.3805, 2.7778, -0.5211, 0.3294, -0.2086, 4.378~
$ gnha_day_no_negative <dbl> 5.3805, 2.7778, 0.0000, 0.3294, 0.0000, 4.3784,~
```

```
$ row_id          <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
$ year           <dbl> 2023, 2023, 2023, 2023, 2023, 2023, 2023, 2023, ~
$ treatment      <chr> "Corn", "Corn", "Sorghum", "Sorghum", "Sorghum"~
```

```
library(dplyr)
library(tidyr)
library(ggplot2)
library(lubridate)

# 1. Prepare Data & Fix Labels
# We factorize 'Treatment' immediately to lock in the "Sorghum + WCC" label order
row_interrow_comparison <- flux_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    Treatment = factor(Treatment, levels = c("Sorghum", "Sorghum + WCC")), # Locks order
    year = year(date)
  ) |>
  filter(year %in% c(2023, 2024)) |>
  select(year, date, Treatment, plot, rowvs_interrow, gnha_day_no_negative)
```

```
# 2. Summary Statistics (Your Request)
row_interrow_summary <- row_interrow_comparison |>
  group_by(year, Treatment, rowvs_interrow) |> # Grouped by Year/Trt/Pos
  summarise(
    mean_flux = mean(gnha_day_no_negative, na.rm = TRUE),
    max_flux = max(gnha_day_no_negative, na.rm = TRUE),
    sd = sd(gnha_day_no_negative, na.rm = TRUE),
    n = sum(!is.na(gnha_day_no_negative)),
    .groups = "drop"
  )
```

```
print("--- Summary Statistics by Position ---")
```

```
[1] "--- Summary Statistics by Position ---"
```

```
print(row_interrow_summary)
```

```
# A tibble: 8 x 7
  year Treatment      rowvs_interrow mean_flux max_flux    sd    n
  <dbl> <fct>          <chr>          <dbl>    <dbl> <dbl> <int>
1  2023 Sorghum      Interrow        5.15    110.  11.3  160
2  2023 Sorghum      Row            7.63    220.  20.9  162
3  2023 Sorghum + WCC Interrow        3.73     59.9   7.35  161
4  2023 Sorghum + WCC Row            6.13    194.  20.7  160
5  2024 Sorghum      Interrow        9.61    241.  30.9  114
6  2024 Sorghum      Row            5.45     73.5  10.1  116
7  2024 Sorghum + WCC Interrow        4.65    148.  14.8  123
8  2024 Sorghum + WCC Row            4.04    128.  12.7  123
```

```
# 3. Calculate Differences for Reviewer MM2 (Interrow - Row)
# This pivots the data wide so we can subtract Row from Interrow for every specific plot/date
diff_analysis <- row_interrow_comparison |>
  pivot_wider(
    names_from = rowvs_interrow,
    values_from = gnha_day_no_negative,
```

```

  names_prefix = "Flux_"
) |>
# Adjust 'Flux_Interrow' / 'Flux_Row' if your actual values in 'rowvs_interrow' differ (e.g. "Row", "Interrow")
mutate(
  Diff = Flux_Interrow - Flux_Row
) |>
filter(!is.na(Diff))

# Statistical Test: Is the mean difference significantly different from 0?
t_test_result <- t.test(diff_analysis$Diff)

print("--- Paired T-Test Results (Reviewer Check) ---")

[1] "--- Paired T-Test Results (Reviewer Check) ---"

print(t_test_result)

```

One Sample t-test

```

data: diff_analysis$Diff
t = -0.5, df = 554, p-value = 0.6
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 -2.10  1.19
sample estimates:
mean of x
 -0.456

```

```

# 4. PLOTS: Split by year for red/blue title bars, swap Row/Interrow order
# Row first (control), Interrow second
row_interrow_comparison <- row_interrow_comparison |>
  mutate(rowvs_interrow = factor(rowvs_interrow, levels = c("Row", "Interrow")))

diff_analysis <- diff_analysis |>
  mutate(year = year(date))

# --- Panel (a): Absolute fluxes, 2023 ---
p_s5a <- ggplot(filter(row_interrow_comparison, year == 2023),
  aes(x = gnha_day_no_negative, color = rowvs_interrow)) +
  geom_boxplot(aes(y = Treatment), outlier.shape = NA, alpha = 0.7) +
  geom_jitter(aes(y = Treatment), position = position_jitterdodge(jitter.width = 0.2),
    alpha = 0.5, size = 0.7) +
  scale_x_log10() +
  annotation_logticks(sides = "b") +
  scale_color_manual(values = c("Row" = "grey80", "Interrow" = "grey40"),
    name = "Position") +
  labs(y = "", x = expression("N"[2]*"0 Flux (g N ha"^-1*" day"^-1*")"),
    tag = "(a)", title = "2023") +
  theme_publication(base_size = 10) +
  theme(legend.position = "top",
    plot.tag = element_text(face = "bold", size = 11),
    plot.tag.position = "topleft",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 12,
      color = "#E41A1C"))

```

```

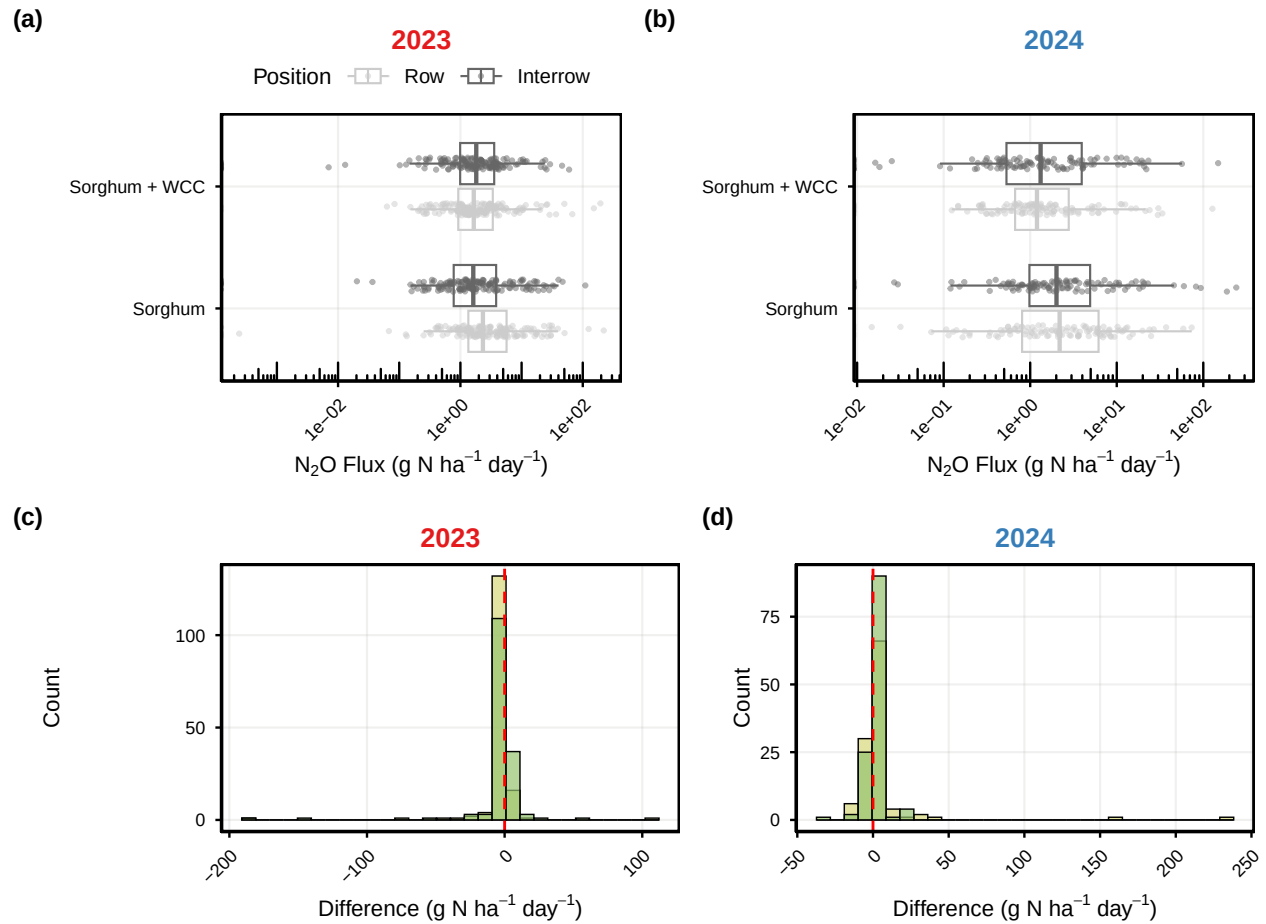
# --- Panel (b): Absolute fluxes, 2024 ---
p_s5b <- ggplot(filter(row_interrow_comparison, year == 2024),
  aes(x = gnha_day_no_negative, color = rowvs_interrow)) +
  geom_boxplot(aes(y = Treatment), outlier.shape = NA, alpha = 0.7) +
  geom_jitter(aes(y = Treatment), position = position_jitterdodge(jitter.width = 0.2),
    alpha = 0.5, size = 0.7) +
  scale_x_log10() +
  annotation_logticks(sides = "b") +
  scale_color_manual(values = c("Row" = "grey80", "Interrow" = "grey40"),
    name = "Position") +
  labs(y = "", x = expression("N"[2]*"0 Flux (g N ha-1* day-1)"),
    tag = "(b)", title = "2024") +
  theme_publication(base_size = 10) +
  theme(legend.position = "none",
    plot.tag = element_text(face = "bold", size = 11),
    plot.tag.position = "topleft",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 12,
      color = "#377EB8"))

# --- Panel (c): Difference histogram, 2023 ---
p_s5c <- ggplot(filter(diff_analysis, year == 2023),
  aes(x = Diff, fill = Treatment)) +
  geom_histogram(bins = 30, alpha = 0.7, position = "identity", color = "black",
    linewidth = 0.3) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red", linewidth = 0.6) +
  scale_fill_manual(values = treatment_colors) +
  labs(x = expression("Difference (g N ha-1* day-1)"),
    y = "Count", tag = "(c)", title = "2023") +
  theme_publication(base_size = 10) +
  theme(legend.position = "none",
    plot.tag = element_text(face = "bold", size = 11),
    plot.tag.position = "topleft",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 12,
      color = "#E41A1C"))

# --- Panel (d): Difference histogram, 2024 ---
p_s5d <- ggplot(filter(diff_analysis, year == 2024),
  aes(x = Diff, fill = Treatment)) +
  geom_histogram(bins = 30, alpha = 0.7, position = "identity", color = "black",
    linewidth = 0.3) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red", linewidth = 0.6) +
  scale_fill_manual(values = treatment_colors) +
  labs(x = expression("Difference (g N ha-1* day-1)"),
    y = "Count", tag = "(d)", title = "2024") +
  theme_publication(base_size = 10) +
  theme(legend.position = "none",
    plot.tag = element_text(face = "bold", size = 11),
    plot.tag.position = "topleft",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 12,
      color = "#377EB8"))

# Display & assemble
rowvsinterrow <- (p_s5a | p_s5b) / (p_s5c | p_s5d)

```



```
# Generate Descriptive Stats for All Treatments
flux_summary_stats <- flux_data |>
# Ensure we capture ALL treatments (remove the filter if Miscanthus is in the raw data)
filter(treatment %in% c("Sorghum", "Sorghum + Rye", "Corn", "Soy")) |>
mutate(
  Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
  year = year(date)
) |>
filter(year %in% c(2023, 2024)) |>
group_by(Treatment) |>
summarise(
  min_flux = min(gnha_day_no_negative, na.rm = TRUE),
  max_flux = max(gnha_day_no_negative, na.rm = TRUE),
  mean_flux = mean(gnha_day_no_negative, na.rm = TRUE),
  median_flux = median(gnha_day_no_negative, na.rm = TRUE),
  n_obs = n()
)

print(flux_summary_stats)
```

```
# A tibble: 4 x 6
```

```
Treatment      min_flux max_flux mean_flux median_flux n_obs
<chr>          <dbl>   <dbl>   <dbl>     <dbl> <int>
```

1 Corn	0	524.	13.2	2.62	535
2 Sorghum	0	241.	6.86	2.02	552
3 Sorghum + WCC	0	194.	4.68	1.38	567
4 Soy	0	249.	5.70	1.75	495

11 Figure 5: Nitrate Leaching and Precipitation

```
# 1. Load and Process Leaching Data
# Define plot area constants
plot_area_ft2 <- 120 * 160
plot_area_ha <- plot_area_ft2 / 107639.1041671

leaching_data <- read_csv("data/clean_leaching.csv") |>
  janitor::clean_names() |>
  mutate(
    date = as.Date(date),
    year = year(date)
  ) |>
  filter(year %in% c(2023, 2024))

# nitrate is in mg_l not mg_ml
leaching_data |>
  group_by(treatment, year, date) |>
  summarise(
    max_nitrate = max(nitrate_mg_l, na.rm = TRUE),
    .groups = "drop"
  ) |>
  slice_max(max_nitrate, by = c(treatment, year))

# A tibble: 8 x 4
  treatment      year date      max_nitrate
  <chr>          <dbl> <date>      <dbl>
1 Corn          2023 2023-06-27    20.2
2 Corn          2024 2024-04-25    28.4
3 Sorghum       2023 2023-05-18    20.4
4 Sorghum       2024 2024-08-16    20.9
5 Sorghum + Rye 2023 2023-06-02    21.2
6 Sorghum + Rye 2024 2024-03-29    14.3
7 Soy           2023 2023-04-21    22.5
8 Soy           2024 2024-07-24    21.0

# 2. Daily Leaching Calculation
daily_leaching_summary <- leaching_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    daily_n_loss_kgha = (total_n_loss_mg / 1e6) / plot_area_ha
  ) |>
  group_by(Treatment, year, date) |>
  summarise(
    mean_loss = mean(daily_n_loss_kgha, na.rm = TRUE),
    se_loss = sd(daily_n_loss_kgha, na.rm = TRUE) / sqrt(sum(!is.na(daily_n_loss_kgha))),
    .groups = "drop"
  )
```

```

# 3. Cumulative Leaching Calculation
# Updated to match Figure 3 R script logic (using SE for ribbons, not 95% CI)
cum_leaching_summary <- leaching_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    cum_loss_kgha = (cumulative_n_loss_mg / 1e6) / plot_area_ha
  ) |>
  group_by(Treatment, year, date) |>
  summarise(
    cum_loss_mean = mean(cum_loss_kgha, na.rm = TRUE),
    cum_loss_sd = sd(cum_loss_kgha, na.rm = TRUE),
    n = n(),
    # Use standard error for ribbons
    cum_loss_se = cum_loss_sd / sqrt(n),
    .groups = "drop"
  )

# 4. Significance Testing (Daily)
daily_leaching_sig <- leaching_data |>
  filter(treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  mutate(
    Treatment = if_else(treatment == "Sorghum + Rye", "Sorghum + WCC", treatment),
    daily_n_loss_kgha = (total_n_loss_mg / 1e6) / plot_area_ha
  ) |>
  group_by(year, date) |>
  summarise(
    p_value = {
      sorg <- daily_n_loss_kgha[Treatment == "Sorghum"]
      wcc <- daily_n_loss_kgha[Treatment == "Sorghum + WCC"]
      if (length(na.omit(sorg)) >= 3 && length(na.omit(wcc)) >= 3) {
        tryCatch(wilcox.test(sorg, wcc)$p.value, error = function(e) NA)
      } else NA
    },
    .groups = "drop"
  ) |>
  mutate(
    star_label = if_else(!is.na(p_value) & p_value < 0.05, "*", "")
  )

# 5. Prepare Plot Data (Stars & Precip)
leaching_stars <- daily_leaching_summary |>
  group_by(year, date) |>
  summarise(max_y = max(mean_loss + se_loss, na.rm = TRUE), .groups = "drop") |>
  inner_join(daily_leaching_sig, by = c("year", "date")) |>
  filter(star_label == "*") |>
  mutate(star_y = max_y * 1.1)

daily_precip_leaching <- rain |>
  filter(year %in% c(2023, 2024)) |>
  group_by(year) |>
  filter(date >= min(leaching_data$date) & date <= max(leaching_data$date)) |>
  ungroup()

```

```

# Scaling factors (match R script logic)
max_leach <- max(daily_leaching_summary$mean_loss + daily_leaching_summary$se_loss, na.rm = TRUE)
max_precip_l <- max(daily_precip_leaching$precipmm, na.rm = TRUE)
scale_factor_l <- (max_leach * 0.4) / max_precip_l
plot_top_l <- max_leach * 1.1

# Updated scaling for ribbons using SE
max_cum_leach <- max(cum_leaching_summary$cum_loss_mean + cum_leaching_summary$cum_loss_se, na.rm = TRUE)

# 6. Plotting Function
create_leaching_plot <- function(year_val, is_right_panel = FALSE) {
  # Data subsets
  d_sub <- filter(daily_leaching_summary, year == year_val)
  c_sub <- filter(cum_leaching_summary, year == year_val)
  p_sub <- filter(daily_precip_leaching, year == year_val)
  s_sub <- filter(leaching_stars, year == year_val)
  fert_year <- filter(fert_events, year == year_val)

  x_lims <- as.Date(paste0(year_val, c("-01-01", "-12-31")))

  # Top Panel: Daily Leaching + Precip
  p_daily <- ggplot() +
    # Precip bars
    geom_rect(
      data = p_sub,
      aes(xmin = date - 0.5, xmax = date + 0.5,
          ymin = plot_top_l - (precipmm * scale_factor_l), ymax = plot_top_l),
      fill = "#377EB8", alpha = 0.7
    ) +
    # Leaching points & errors
    geom_point(data = d_sub, aes(x = date, y = mean_loss, color = Treatment), size = 1.5) +
    geom_errorbar(
      data = d_sub,
      aes(x = date, ymin = mean_loss - se_loss, ymax = mean_loss + se_loss, color = Treatment),
      width = 0.2, alpha = 0.5
    ) +
    geom_vline(
      data = fert_year,
      aes(xintercept = fertilizer_date),
      linetype = "dashed",
      color = "black",
      alpha = 0.7
    ) +
    # Significance stars
    geom_text(data = s_sub, aes(x = date, y = star_y, label = star_label), size = 5) +
    scale_color_manual(values = treatment_colors) +
    guides(color = "none") +
    scale_x_date(date_breaks = "2 months", date_labels = "%b", limits = x_lims) +
    scale_y_continuous(
      name = if(!is_right_panel) expression("Daily N Loss (kg N ha-1 d-1)") else "",
      limits = c(0, plot_top_l),
      sec.axis = sec_axis(~ (plot_top_l - .) / scale_factor_l,

```

```

      name = if(is_right_panel) "Precipitation (mm)" else "")
) +
theme_publication(base_size = 10) +
theme(
  legend.position = "none",
  axis.text.x = element_blank(),
  axis.ticks.x = element_blank(),
  axis.line.x = element_blank(),
  axis.text.y = if (is_right_panel) element_blank() else element_text(),
  axis.ticks.y = if (is_right_panel) element_blank() else element_line(),
  axis.line.y = if (is_right_panel) element_blank() else element_line(),
  axis.title.y = if (is_right_panel) element_blank() else element_text(),
  axis.text.y.right = if (is_right_panel) {
    element_text(color = "#377EB8")
  } else {
    element_blank()
  },
  axis.ticks.y.right = if (is_right_panel) {
    element_line()
  } else {
    element_blank()
  },
  axis.title.y.right = if (is_right_panel) {
    element_text(color = "#377EB8")
  } else {
    element_blank()
  }
) +
labs(x = "") +
geom_hline(yintercept = 0, linetype = "solid", alpha = 0.3)

if(is_right_panel) {
  p_daily <- p_daily + theme(
    axis.text.y.left = element_blank(),
    axis.ticks.y.left = element_blank(),
    axis.line.y.left = element_blank()
  )
} else {
  p_daily <- p_daily + theme(
    axis.text.y.right = element_blank(),
    axis.ticks.y.right = element_blank()
  )
}

# Bottom Panel: Cumulative Leaching (Updated)
p_cum <- ggplot(c_sub, aes(x = date, y = cum_loss_mean, color = Treatment, fill = Treatment)) +
  geom_ribbon(
    # CHANGED: Using SE instead of CI to match R script
    aes(ymin = pmax(cum_loss_mean - cum_loss_se, 0), ymax = cum_loss_mean + cum_loss_se),
    alpha = 0.15, linewidth = 0
  ) +
  geom_line(linewidth = 1) +
  geom_vline(

```

```

    data = fert_year,
    aes(xintercept = fertilizer_date),
    linetype = "dashed",
    color = "black",
    alpha = 0.7
  ) +
  scale_color_manual(values = treatment_colors) +
  scale_fill_manual(values = treatment_colors) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b", limits = x_lims) +
  scale_y_continuous(
    name = if(!is_right_panel) expression("Cumulative N Loss (kg N ha"^-1*")") else "",
    limits = c(0, max_cum_leach)
  ) +
  labs(x = "Month") +
  theme_publication(base_size = 10) +
  theme(legend.position = "none")

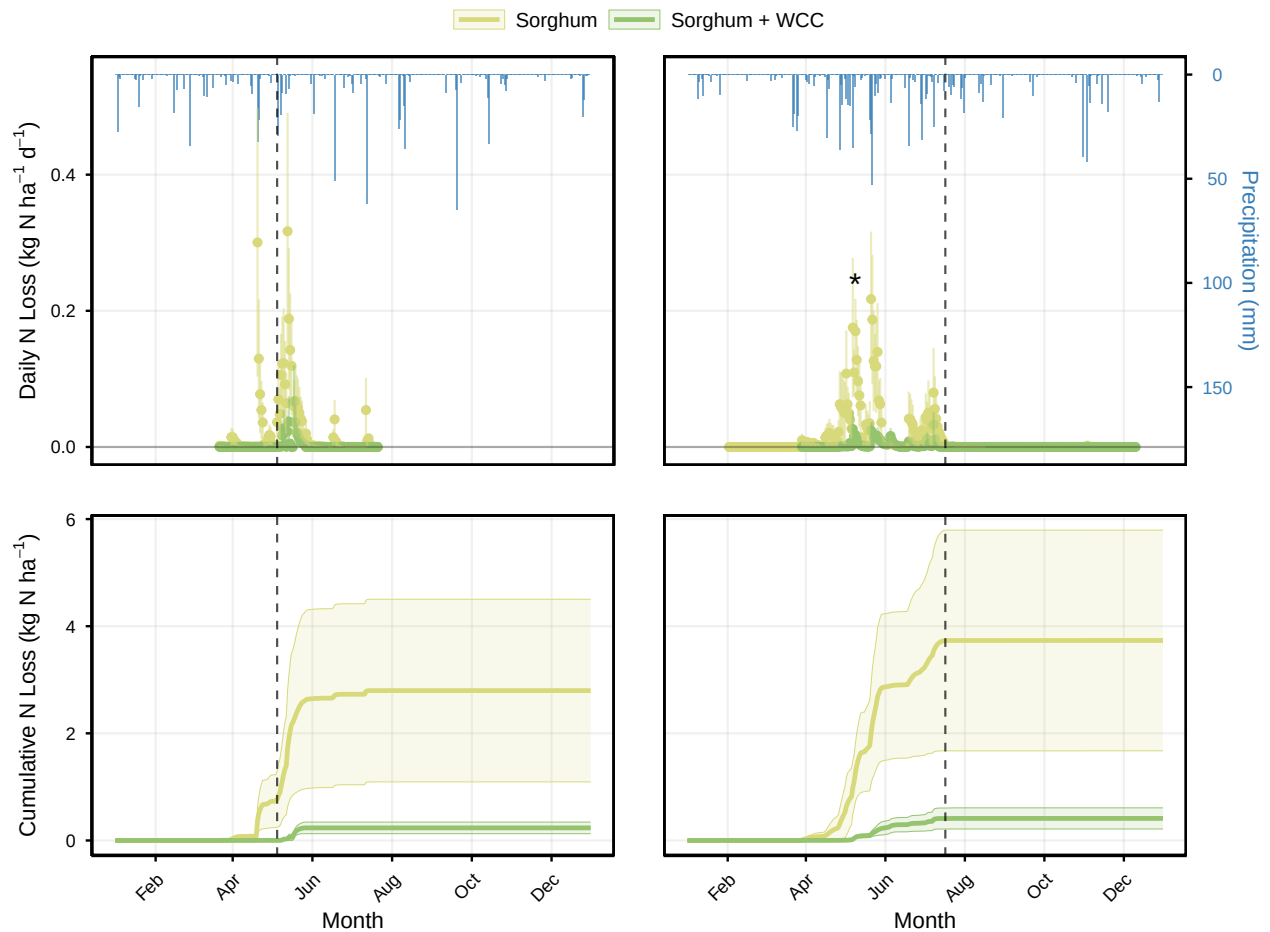
if(is_right_panel) {
  p_cum <- p_cum + theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    axis.line.y = element_blank()
  )
}

p_daily / p_cum + plot_layout(heights = c(1.2, 1))
}

# 7. Assemble Figure
fig_leaching <- (create_leaching_plot(2023) | create_leaching_plot(2024, TRUE)) +
  plot_layout(guides = "collect") &
  theme(legend.position = "top", legend.justification = "center") &
  labs(color = NULL, fill = NULL)

fig_leaching

```



```
ggsave("figures/figure3_leaching_precip.png", fig_leaching,
       width = 174, height = 174, units = "mm", dpi = 600, bg = "white")
```

```
# Prepare concentration data
conc_data <- leaching_data |>
  filter(!is.na(nitrate_mg_l), !is.na(ammonia_mg_l)) |>
  mutate(
    total_inorganic_n = nitrate_mg_l + ammonia_mg_l,
    prop_nitrate = nitrate_mg_l / total_inorganic_n
  ) |>
  filter(total_inorganic_n > 0) # Avoid division issues

# All treatments with renamed labels and order
conc_all <- conc_data |>
  mutate(
    Treatment = case_when(
      treatment == "Corn" ~ "Maize",
      treatment == "Soy" ~ "Soybean",
      treatment == "Sorghum + Rye" ~ "Sorghum + WCC",
      TRUE ~ treatment
    ),
    Treatment = factor(Treatment,
                      levels = c("Maize", "Soybean", "Sorghum", "Sorghum + WCC"))
  )
```

```

# Also keep Sorghum vs WCC subset for t-test
conc_sr <- conc_all |>
  filter(Treatment %in% c("Sorghum", "Sorghum + WCC"))

# Medians for vertical lines
conc_medians <- conc_all |>
  group_by(Treatment) |>
  summarise(median_no3 = median(nitrate_mg_l, na.rm = TRUE), .groups = "drop")

# t-test on mean concentrations (Sorghum vs WCC)
ttest <- t.test(nitrate_mg_l ~ Treatment, data = conc_sr)
ttest_table <- tibble(
  comparison = "Sorghum vs Sorghum + WCC",
  mean_sorghum = mean(conc_sr$nitrate_mg_l[conc_sr$Treatment == "Sorghum"], na.rm = TRUE),
  mean_wcc = mean(conc_sr$nitrate_mg_l[conc_sr$Treatment == "Sorghum + WCC"], na.rm = TRUE),
  diff_means = mean_sorghum - mean_wcc,
  t_stat = unname(ttest$statistic),
  df = unname(ttest$parameter),
  p_value = ttest$p.value
)

# Subtle colors for Maize/Soybean
s4_colors <- c("Maize"      = "#B0B0B0",
               "Soybean"    = "#A8C8E8",
               "Sorghum"    = "#D8D97AFF",
               "Sorghum + WCC" = "#95C36EFF")

# Panel (a): Density plot - all treatments
p_s4a <- ggplot(conc_all, aes(x = nitrate_mg_l, fill = Treatment, color = Treatment)) +
  geom_density(aes(y = after_stat(density * 100)), alpha = 0.5, linewidth = 0.3) +
  geom_vline(data = conc_medians, aes(xintercept = median_no3, color = Treatment),
            linetype = "dashed", linewidth = 0.8) +
  labs(
    tag = "(a)",
    x = expression(NO[3]^{-1} * N Concentration (mg L^{-1} *)),
    y = "Proportion of Measurements (%)",
    fill = "Treatment", color = "Treatment"
  ) +
  scale_fill_manual(values = s4_colors) +
  scale_color_manual(values = s4_colors) +
  theme_publication(base_size = 10) +
  theme(legend.position = "top",
        plot.tag = element_text(face = "bold", size = 11),
        plot.tag.position = "topleft")

# Panel (b): Proportion nitrate boxplots - all treatments, y trimmed to 50% min
p_s4b <- ggplot(conc_all, aes(x = Treatment, y = prop_nitrate * 100, fill = Treatment)) +
  geom_boxplot(outlier.shape = 1, alpha = 0.7, color = "black") +
  geom_hline(yintercept = 90, linetype = "dashed", color = "grey50") +
  coord_cartesian(ylim = c(50, 100)) +
  scale_fill_manual(values = s4_colors) +
  labs(
    tag = "(b)",

```

Comparison	Mean NO3 (Sorghum)	Mean NO3 (Sorghum + WCC)	Mean Difference
Sorghum vs Sorghum + WCC	7.32	3.5	3.8

```

    x = "Treatment",
    y = expression("NO3-"*"-N as % of Total Inorganic N")
  ) +
  theme_publication(base_size = 10) +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 0, hjust = 0.5),
        plot.tag = element_text(face = "bold", size = 11),
        plot.tag.position = "topleft")

# Combine panels
fig_s4 <- p_s4a / p_s4b

# T-test table
ttest_table |>
  mutate(
    mean_sorghum = round(mean_sorghum, 3),
    mean_wcc = round(mean_wcc, 3),
    diff_means = round(diff_means, 3),
    t_stat = round(t_stat, 3),
    df = round(df, 1),
    p_value = ifelse(p_value < 0.001, "< 0.001", sprintf("%.3f", p_value))
  ) |>
  gt() |>
  cols_label(
    comparison = "Comparison",
    mean_sorghum = "Mean NO3 (Sorghum)",
    mean_wcc = "Mean NO3 (Sorghum + WCC)",
    diff_means = "Mean Difference",
    t_stat = "t",
    df = "df",
    p_value = "p"
  )

# Save
ggsave("figures/FigureS4_nitrate_concentration_proportion.tiff", fig_s4,
       width = 174, height = 150, units = "mm", dpi = 600, bg = "white",
       compression = "lzw")

# Measurement counts by year and plot (concentration data only)
# Ensure year exists
if (!("year" %in% names(conc_data))) {
  conc_data <- conc_data |>
    mutate(date = as.Date(date), year = lubridate::year(date))
}

conc_counts_by_year_plot <- conc_data |>
  group_by(year, treatment, plot) |>
  summarise(n_measurements = n(), .groups = "drop") |>
  arrange(year, treatment, plot)

```

Year	Treatment	Plot	N Measurements
2023	Corn	01	13
2023	Corn	08	14
2023	Corn	11	13
2023	Corn	14	12
2023	Sorghum	02	13
2023	Sorghum	03	10
2023	Sorghum	05	7
2023	Sorghum	15	12
2023	Sorghum + Rye	07	2
2023	Sorghum + Rye	09	13
2023	Sorghum + Rye	12	2
2023	Sorghum + Rye	13	9
2023	Soy	04	8
2023	Soy	06	17
2023	Soy	10	14
2024	Corn	04	14
2024	Corn	06	20
2024	Corn	10	19
2024	Sorghum	02	19
2024	Sorghum	03	16
2024	Sorghum	05	17
2024	Sorghum	15	16
2024	Sorghum + Rye	07	11
2024	Sorghum + Rye	09	20
2024	Sorghum + Rye	12	12
2024	Sorghum + Rye	13	17
2024	Soy	01	20
2024	Soy	08	22
2024	Soy	11	16
2024	Soy	14	18

```

conc_counts_by_year_plot |>
  gt() |>
  cols_label(
    year = "Year",
    treatment = "Treatment",
    plot = "Plot",
    n_measurements = "N Measurements"
  ) |>
  cols_align(align = "center", columns = -treatment)

```

```

# Summary counts by year and treatment
conc_counts_by_year_treatment <- conc_data |>
  group_by(year, treatment) |>
  summarise(n_measurements = n(), .groups = "drop") |>

```

Year	Treatment	N Measurements
2023	Corn	52
2023	Sorghum	42
2023	Sorghum + Rye	26
2023	Soy	39
2024	Corn	53
2024	Sorghum	68
2024	Sorghum + Rye	60
2024	Soy	76

```

arrange(year, treatment)

conc_counts_by_year_treatment |>
  gt() |>
  cols_label(
    year = "Year",
    treatment = "Treatment",
    n_measurements = "N Measurements"
  ) |>
  cols_align(align = "center", columns = -treatment)

```

12 Figure S_NEW: Stacked Bar — NH₄/NO₃ Proportions Over Time

```

# Prepare time-series of NH4 vs NO3 proportions by treatment and sampling date
nh4_no3_ts <- conc_data |>
  mutate(
    date = as.Date(date),
    year = lubridate::year(date),
    Treatment = case_when(
      treatment == "Corn" ~ "Maize",
      treatment == "Soy" ~ "Soybean",
      treatment == "Sorghum + Rye" ~ "Sorghum + WCC",
      TRUE ~ treatment
    ),
    Treatment = factor(Treatment,
      levels = c("Maize", "Soybean", "Sorghum", "Sorghum + WCC"))
  ) |>
  filter(year %in% c(2023, 2024)) |>
  group_by(Treatment, year, date) |>
  summarise(
    mean_no3 = mean(nitrate_mg_l, na.rm = TRUE),
    mean_nh4 = mean(ammonia_mg_l, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(total = mean_no3 + mean_nh4,
    pct_no3 = mean_no3 / total * 100,
    pct_nh4 = mean_nh4 / total * 100) |>

```

```

pivot_longer(cols = c(pct_no3, pct_nh4),
              names_to = "N_form", values_to = "pct") |>
mutate(
  N_form = if_else(N_form == "pct_no3",
                   "NO\u2083\u207B-N", "NH\u2084\u207A-N"),
  N_form = factor(N_form, levels = c("NH\u2084\u207A-N", "NO\u2083\u207B-N"))
)

# Standardize dates to a common year for overlay
nh4_no3_ts <- nh4_no3_ts |>
  mutate(plot_date = as.Date(paste0("2000-", format(date, "%m-%d"))))

# Colors for N forms
n_form_colors <- c("NO\u2083\u207B-N" = "#4393C3", # blue
                  "NH\u2084\u207A-N" = "#D6604D") # red-orange

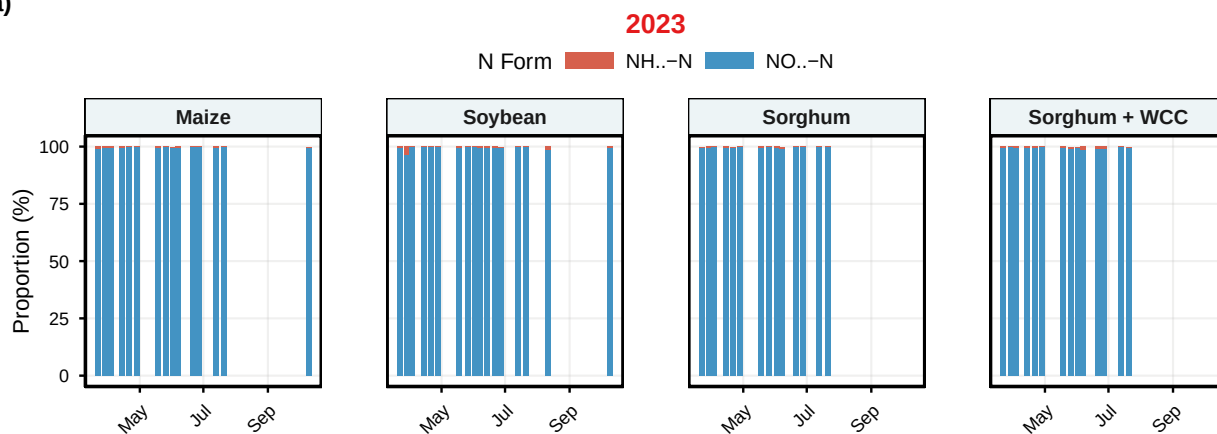
# Stacked bar per year
make_nh4no3_panel <- function(yr, tag_label, year_color) {
  df <- filter(nh4_no3_ts, year == yr)
  ggplot(df, aes(x = plot_date, y = pct, fill = N_form)) +
    geom_col(position = "stack", width = 5, color = NA) +
    facet_wrap(~ Treatment, nrow = 1) +
    scale_x_date(date_breaks = "2 months", date_labels = "%b") +
    scale_fill_manual(values = n_form_colors, name = "N Form") +
    labs(x = "", y = "Proportion (%)", tag = tag_label,
         title = as.character(yr)) +
    coord_cartesian(ylim = c(0, 100)) +
    theme_publication(base_size = 10) +
    theme(
      legend.position = if (tag_label == "(a)") "top" else "none",
      plot.tag = element_text(face = "bold", size = 11),
      plot.tag.position = "topleft",
      plot.title = element_text(hjust = 0.5, face = "bold", size = 12,
                                color = year_color),
      strip.text = element_text(size = 9, face = "bold")
    )
}

p_nh4no3_a <- make_nh4no3_panel(2023, "(a)", "#E41A1C")
p_nh4no3_b <- make_nh4no3_panel(2024, "(b)", "#377EB8")

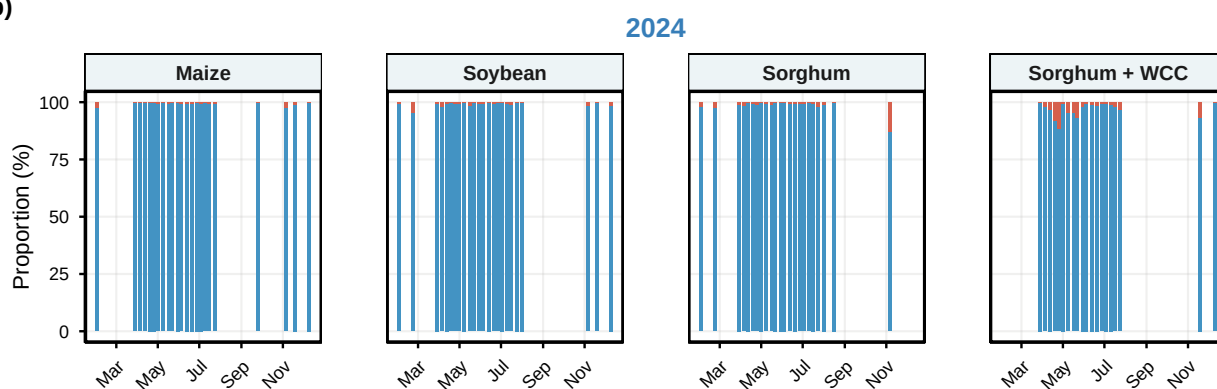
fig_nh4no3 <- p_nh4no3_a / p_nh4no3_b
fig_nh4no3

```

(a)



(b)



```
ggsave("figures/FigureS_NH4_NO3_proportions.tiff", fig_nh4no3,
        width = 174, height = 130, units = "mm", dpi = 600, bg = "white",
        compression = "lzw")
```

13 Table 3: Annual Nitrogen Losses

```
library(gt)
library(emmeans)
library(multcomp)

# --- 1. N2O Data Processing (Interpolation & Cumulative) ---

# Summarize daily flux by plot/date
daily_flux_summary <- flux_data |>
  group_by(plot, rowvs_interrow, treatment, year, date) |>
  summarise(mean_flux = mean(gnha_day_no_negative, na.rm = TRUE), .groups = "drop") |>
  arrange(plot, rowvs_interrow, treatment, year, date)

# Interpolation helper function
interpolate_daily_flux <- function(data) {
  # Get unique plot combinations
  plot_combinations <- data |>
    distinct(plot, rowvs_interrow, treatment, year)
```

```

purrr::map_dfr(seq_len(nrow(plot_combinations)), function(i) {
  combo <- plot_combinations[i, ]

  subset_data <- data |>
    filter(
      plot == combo$plot,
      rowvs_interrow == combo$rowvs_interrow,
      treatment == combo$treatment,
      year == combo$year
    ) |>
    arrange(date)

  # Create full year sequence
  start_date <- as.Date(paste0(combo$year, "-01-01"))
  end_date <- as.Date(paste0(combo$year, "-12-31"))
  all_days <- seq(start_date, end_date, by = "day")

  # Interpolate if we have sufficient data
  if (nrow(subset_data) >= 2 && !all(is.na(subset_data$mean_flux))) {
    valid_data <- subset_data[!is.na(subset_data$mean_flux), ]

    if (nrow(valid_data) >= 2) {
      interpolated_flux <- approx(
        x = as.numeric(valid_data$date),
        y = valid_data$mean_flux,
        xout = as.numeric(all_days),
        rule = 1
      )$y
    } else {
      interpolated_flux <- rep(NA_real_, length(all_days))
    }
  } else {
    interpolated_flux <- rep(NA_real_, length(all_days))
  }

  tibble(
    plot = combo$plot,
    rowvs_interrow = combo$rowvs_interrow,
    treatment = combo$treatment,
    year = combo$year,
    date = all_days,
    interpolated_flux = interpolated_flux
  )
})
}

# Apply interpolation
interpolated_flux_data <- interpolate_daily_flux(daily_flux_summary)

# Calculate cumulative N2O per plot (g -> kg)
plot_cumulative_n2o <- interpolated_flux_data |>
  group_by(plot, rowvs_interrow, treatment, year) |>
  summarise(

```

```

    annual_cumulative_n2o = sum(interpolated_flux, na.rm = TRUE) / 1000,
    .groups = "drop"
  ) |>
  filter(is.finite(annual_cumulative_n2o))

# --- 2. Leaching Data Processing ---

# Ensure leaching data is available (re-load if necessary)
if (!exists("leaching_data")) {
  leaching_data <- read_csv("data/clean_leaching.csv") |>
    janitor::clean_names() |>
    mutate(
      date = as.Date(date),
      year = year(date)
    ) |>
    filter(year %in% c(2023, 2024))
}

# Calculate annual cumulative nitrogen leaching per plot
plot_area_ft2 <- 120 * 160
plot_area_ha <- plot_area_ft2 / 107639.1041671

annual_leaching <- leaching_data |>
  group_by(plot, treatment, year) |>
  summarise(
    # mg -> kg, then divide by hectares
    annual_cumulative_leaching = (max(cumulative_n_loss_mg, na.rm = TRUE) / 1e6) / plot_area_ha,
    .groups = "drop",
    annual_drainage_l = max(cumulative_flow_l, na.rm = TRUE),
    annual_drainage_mm = (annual_drainage_l / plot_area_ha) / 10000
  ) |>
  filter(is.finite(annual_cumulative_leaching), annual_cumulative_leaching >= 0)

# --- 3. Statistical Summaries and Letters ---

# Summarize by treatment and year
n2o_summary <- plot_cumulative_n2o |>
  group_by(treatment, year) |>
  summarise(
    n_plots = n(),
    mean_n2o = mean(annual_cumulative_n2o, na.rm = TRUE),
    sd_n2o = sd(annual_cumulative_n2o, na.rm = TRUE),
    .groups = "drop"
  )

leaching_summary <- annual_leaching |>
  group_by(treatment, year) |>
  summarise(
    n_plots = n(),
    mean_leaching = mean(annual_cumulative_leaching, na.rm = TRUE),
    sd_leaching = sd(annual_cumulative_leaching, na.rm = TRUE),
    .groups = "drop"
  )

```

```

# Helper: Compute Tukey letters within year
compute_letters_by_year <- function(df, value_col) {
  d <- df |> filter(!is.na(.data[[value_col]]))
  trts <- sort(unique(d$treatment))
  if (length(trts) < 2) return(tibble(treatment = trts, letter = "a"))

  fml <- as.formula(paste(value_col, "~ treatment"))
  fit <- lm(fml, data = d)
  emm <- emmeans::emmeans(fit, ~ treatment)

  if (requireNamespace("multcomp", quietly = TRUE)) {
    out <- multcomp::cld(emm, adjust = "tukey", Letters = letters) |>
      as_tibble() |>
      transmute(treatment, letter = gsub("\\s+", "", .group)) |>
      mutate(
        letter = ifelse(grepl("[0-9]", letter), chartr("123456789", "abcdefghi", letter), letter),
        letter = ifelse(letter == "" | is.na(letter), "a", letter)
      )
  } else {
    out <- tibble(treatment = trts, letter = "a")
  }
  out
}

# Calculate letters
leach_letters <- annual_leaching |>
  group_by(year) |>
  group_modify(~ compute_letters_by_year(.x, "annual_cumulative_leaching")) |>
  ungroup()

n2o_letters <- plot_cumulative_n2o |>
  group_by(year) |>
  group_modify(~ compute_letters_by_year(.x, "annual_cumulative_n2o")) |>
  ungroup()

# Join letters (only show for annual rows, not 2-year total later)
leaching_summary <- leaching_summary |>
  left_join(leach_letters, by = c("treatment", "year")) |>
  mutate(leaching_result = sprintf("%.3f ± %.3f (%s)", mean_leaching, sd_leaching, letter))

n2o_summary <- n2o_summary |>
  left_join(n2o_letters, by = c("treatment", "year")) |>
  mutate(n2o_result = sprintf("%.3f ± %.3f (%s)", mean_n2o, sd_n2o, letter))

# Calculate 2-Year Totals (Sum of Means, SD via Quadrature)
n2o_two_year <- n2o_summary |>
  filter(year %in% c(2023, 2024)) |>
  group_by(treatment) |>
  summarise(
    mean_n2o = sum(mean_n2o, na.rm = TRUE),
    sd_n2o = sqrt(sum(sd_n2o^2, na.rm = TRUE)),
    .groups = "drop"
  ) |>

```

```

    mutate(year = "2023-2024", n2o_result = sprintf("%.3f ± %.3f", mean_n2o, sd_n2o))

leaching_two_year <- leaching_summary |>
  filter(year %in% c(2023, 2024)) |>
  group_by(treatment) |>
  summarise(
    mean_leaching = sum(mean_leaching, na.rm = TRUE),
    sd_leaching = sqrt(sum(sd_leaching^2, na.rm = TRUE)),
    .groups = "drop"
  ) |>
  mutate(year = "2023-2024", leaching_result = sprintf("%.3f ± %.3f", mean_leaching, sd_leaching))

# --- 4. Organize Data for Table ---

# Extract numeric means for Cost calculation
n2o_means <- bind_rows(
  n2o_summary |> mutate(year = as.character(year)) |> select(year, treatment, mean_n2o),
  n2o_two_year |> select(year, treatment, mean_n2o)
)

leaching_means <- bind_rows(
  leaching_summary |> mutate(year = as.character(year)) |> select(year, treatment, mean_leaching),
  leaching_two_year |> select(year, treatment, mean_leaching)
)

# Combine textual results
combined_nitrogen_data <- bind_rows(
  n2o_summary |> mutate(year = as.character(year)) |> select(year, treatment, n2o_result),
  n2o_two_year |> select(year, treatment, n2o_result)
) |>
  full_join(
    bind_rows(
      leaching_summary |> mutate(year = as.character(year)) |> select(year, treatment, leaching_result),
      leaching_two_year |> select(year, treatment, leaching_result)
    ),
    by = c("year", "treatment")
  ) |>
  left_join(n2o_means, by = c("year", "treatment")) |>
  left_join(leaching_means, by = c("year", "treatment")) |>
  mutate(
    year_order = case_when(year == "2023" ~ 1, year == "2024" ~ 2, year == "2023-2024" ~ 3),
    # Preza-Fontes cost values
    environmental_cost_dollars = (mean_n2o * 16.18) + (mean_leaching * 18.54),
    total_environmental_cost = sprintf("$%.2f", environmental_cost_dollars)
  )

# ANOVA / Source of Variation
combined_n2o_data <- plot_cumulative_n2o |> mutate(year_factor = as.factor(year))
aov_n2o <- aov(annual_cumulative_n2o ~ year_factor * treatment, data = combined_n2o_data)

combined_leaching_data <- annual_leaching |> mutate(year_factor = as.factor(year))
aov_leaching <- aov(annual_cumulative_leaching ~ year_factor * treatment, data = combined_leaching_data)

```

```

source_variation_n <- tibble(
  source = c("Year", "Treatment", "Year × Treatment"),
  n2o_p = summary(aov_n2o)[[1]][1:3, "Pr(>F)"],
  leaching_p = summary(aov_leaching)[[1]][1:3, "Pr(>F)"]
) |>
mutate(
  across(c(n2o_p, leaching_p), ~ case_when(
    .x < 0.001 ~ "< 0.001",
    .x < 0.01 ~ sprintf("%.3f", .x),
    TRUE ~ sprintf("%.2f", .x)
  )),
  year = "", year_order = 4, treatment = source,
  n2o_result = n2o_p, leaching_result = leaching_p,
  total_environmental_cost = ""
) |>
select(-source, -n2o_p, -leaching_p)

# Assemble Complete Table
separator_n <- tibble(
  year = "Source of Variation", year_order = 3.5, treatment = "",
  n2o_result = "", leaching_result = "", total_environmental_cost = ""
)

table3_complete <- bind_rows(
  combined_nitrogen_data, separator_n, source_variation_n
) |>
arrange(year_order, treatment) |>
mutate(
  year_display = case_when(
    year == "Source of Variation" ~ year,
    year != lag(year, default = "") ~ as.character(year),
    TRUE ~ ""
  ),
  is_new_section = year != lag(year, default = "") & year != "Source of Variation"
)

# --- 5. Generate GT Table ---

table3_gt <- table3_complete |>
select(
  Year = year_display,
  Treatment = treatment,
  `N2O Loss` = n2o_result,
  `N Leaching` = leaching_result,
  `Total Environmental Cost` = total_environmental_cost,
  is_new_section
) |>
gt() |>
tab_header(title = md("**Table 3. Annual nitrogen losses**")) |>
cols_label(
  `N2O Loss` = md("N<sub>2</sub>O Loss<br>(kg N ha<sup>-1</sup>)"),
  `N Leaching` = md("N Leaching<br>(kg N ha<sup>-1</sup>)"),
  `Total Environmental Cost` = md("Total Environmental Cost<br>(USD ha<sup>-1</sup>)")
)

```

```

) |>
tab_style(
  style = cell_borders(sides = "top", weight = px(1), color = "gray"),
  locations = cells_body(rows = is_new_section == TRUE)
) |>
cols_width(
  Year ~ px(90), Treatment ~ px(150),
  `N2O Loss` ~ px(160), `N Leaching` ~ px(160),
  `Total Environmental Cost` ~ px(160)
) |>
tab_style(
  style = cell_borders(sides = "top", weight = px(2)),
  locations = cells_body(rows = Year == "Source of Variation")
) |>
tab_style(
  style = cell_text(weight = "bold"),
  locations = cells_body(rows = Year == "Source of Variation")
) |>
cols_hide(columns = is_new_section) |>
tab_footnote(
  footnote = "Values are mean ± standard deviation. Letters (in parentheses) indicate Tukey-adjusted p-values."
)

# print table
table3_gt

library(dplyr)
library(tidyr)
library(gt)
library(emmeans)
library(multcomp)

# --- 1. PREPARE PLOT-LEVEL DATA (JOINING N2O & LEACHING) ---

# A. Aggregate N2O to single Plot value (averaging row/interrow)
n2o_plot_clean <- plot_cumulative_n2o |>
  group_by(year, plot, treatment) |>
  summarise(
    n2o_kg_ha = mean(annual_cumulative_n2o, na.rm = TRUE),
    .groups = "drop"
  )

# B. Prepare Leaching (already plot level)
leach_plot_clean <- annual_leaching |>
  select(year, plot, treatment, leaching_kg_ha = annual_cumulative_leaching)

# C. Join and Calculate Plot-Level EDC
# Costs: N2O = $16.18/kg N, Leaching = $18.54/kg N (Preza-Fontes et al.)
plot_level_economics <- n2o_plot_clean |>
  inner_join(leach_plot_clean, by = c("year", "plot", "treatment")) |>
  mutate(
    edc_dollars = (n2o_kg_ha * 16.18) + (leaching_kg_ha * 18.54)
  )

# --- 2. PREPARE "2-YEAR TOTAL" DATA FOR STATS ---

```

Table 3. Annual nitrogen losses

Year	Treatment	N ₂ O Loss (kg N ha ⁻¹)	N Leaching (kg N ha ⁻¹)	Total Environmental Cost (USD ha ⁻¹)
2023	Corn	1.890 ± 1.335 (a)	7.256 ± 3.395 (b)	\$165.10
	Sorghum	1.228 ± 0.808 (a)	2.798 ± 3.410 (ab)	\$71.75
	Sorghum + Rye	0.996 ± 0.644 (a)	0.235 ± 0.217 (a)	\$20.48
	Soy	1.439 ± 0.625 (a)	2.213 ± 3.431 (ab)	\$64.32
2024	Corn	2.418 ± 1.473 (a)	5.730 ± 3.335 (ab)	\$145.35
	Sorghum	1.608 ± 1.103 (a)	3.734 ± 4.120 (ab)	\$95.24
	Sorghum + Rye	0.868 ± 0.546 (a)	0.411 ± 0.395 (a)	\$21.68
	Soy	1.354 ± 1.400 (a)	16.614 ± 12.106 (b)	\$329.93
2023-2024	Corn	4.308 ± 1.988	12.986 ± 4.759	\$310.45
	Sorghum	2.836 ± 1.368	6.532 ± 5.348	\$166.98
	Sorghum + Rye	1.865 ± 0.844	0.646 ± 0.450	\$42.15
	Soy	2.793 ± 1.534	18.827 ± 12.583	\$394.25
Source of Variation				
	Treatment	0.03	0.009	
	Year	0.66	0.08	
	Year × Treatment	0.78	0.04	

Values are mean ± standard deviation. Letters (in parentheses) indicate Tukey-adjusted groups within year (= 0.05). Two-year totals (2023–2024) are shown in Table 3: Annual Nitrogen Losses

```

# We sum the 2023 and 2024 values for each plot to test the "Combined" effect
plot_level_2year <- plot_level_economics |>
  filter(year %in% c(2023, 2024)) |>
  group_by(plot, treatment) |>
  summarise(
    n2o_kg_ha = sum(n2o_kg_ha, na.rm = TRUE),
    leaching_kg_ha = sum(leaching_kg_ha, na.rm = TRUE),
    edc_dollars = sum(edc_dollars, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(year = "2023-2024") # Label for the table

# Combine Annual and 2-Year Data into one master dataset for stats
master_stats_data <- bind_rows(
  plot_level_economics |> mutate(year = as.character(year)),
  plot_level_2year
)

# --- 3. STATS LOOP (Calculate Means + Letters for ALL 3 Columns) ---

# Helper function: Linear Model + Tukey Letters
get_stats_with_letters <- function(df, val_col) {
  # 1. Calculate Means & SD
  summ <- df |>
    group_by(treatment) |>
    summarise(
      mean_val = mean(.data[[val_col]], na.rm = TRUE),
      sd_val = sd(.data[[val_col]], na.rm = TRUE),
      .groups = "drop"
    )

  # 2. Generate Letters (LM -> Emmeans -> CLD)
  # Fit model
  fit <- lm(as.formula(paste(val_col, "~ treatment")), data = df)

  # Get letters
  letters_df <- tryCatch({
    fit |>
      emmeans(~ treatment) |>
      cld(adjust = "tukey", Letters = letters, alpha = 0.05) |>
      as_tibble() |>
      mutate(letter = trimws(.group)) |>
      select(treatment, letter)
  }, error = function(e) {
    tibble(treatment = unique(df$treatment), letter = "")
  })

  # 3. Join
  left_join(summ, letters_df, by = "treatment")
}

# Run the loop for each Year (2023, 2024, 2023-2024) and each Variable
final_table_data <- master_stats_data |>

```

```

group_by(year) |>
group_modify(~ {
  # N2O Stats
  n2o <- get_stats_with_letters(.x, "n2o_kg_ha") |>
  mutate(n2o_str = sprintf("%.2f \u00B1 %.2f (%s)", mean_val, sd_val, letter)) |>
  select(treatment, n2o_str)

  # Leaching Stats
  leach <- get_stats_with_letters(.x, "leaching_kg_ha") |>
  mutate(leach_str = sprintf("%.2f \u00B1 %.2f (%s)", mean_val, sd_val, letter)) |>
  select(treatment, leach_str)

  # EDC Stats (The new addition!)
  edc <- get_stats_with_letters(.x, "edc_dollars") |>
  mutate(edc_str = sprintf("$%.2f \u00B1 %.2f (%s)", mean_val, sd_val, letter)) |>
  select(treatment, edc_str)

  # Merge them all
  n2o |> left_join(leach, by="treatment") |> left_join(edc, by="treatment")
}) |>
ungroup()

# --- 4. FORMATTING THE GT TABLE ---

# Re-calculate ANOVA P-values for the footer (Global Model)
# Note: This is just for the footer P-values
aov_edc <- aov(edc_dollars ~ as.factor(year) * treatment, data = plot_level_economics)
p_trt_edc <- summary(aov_edc)[[1]]["treatment", "Pr(>F)"]

final_table_data |>
mutate(
  year_order = case_when(year == "2023" ~ 1, year == "2024" ~ 2, TRUE ~ 3)
) |>
arrange(year_order, treatment) |>
select(Year = year, Treatment = treatment, `N2O Loss` = n2o_str, `N Leaching` = leach_str, `Env. Damage` = edc_str) |>
gt() |>
tab_header(title = md("**Table 3. Annual nitrogen losses and environmental costs**")) |>
cols_label(
  `N2O Loss` = md("N<sub>2</sub>O Loss<br>(kg N ha<sup>-1</sup>)"),
  `N Leaching` = md("N Leaching<br>(kg N ha<sup>-1</sup>)"),
  `Env. Damage Cost` = md("Total Environmental Cost<br>(USD ha<sup>-1</sup>)")
) |>
cols_align(align = "center", columns = -Treatment) |>
tab_style(
  style = cell_borders(sides = "top", color = "gray"),
  locations = cells_body(rows = Year != lag(Year, default = ""))
) |>
tab_footnote(
  footnote = md(sprintf(
    "Values are mean ± SD. Letters indicate Tukey-adjusted groups within year/period (=0.05). Costs:
    p_trt_edc
  ))
)

```

Table 3. Annual nitrogen losses and environmental costs

Year	Treatment	N ₂ O Loss (kg N ha ⁻¹)
2023	Corn	1.89 ± 0.81 (a)
2023	Sorghum	1.23 ± 0.68 (a)
2023	Sorghum + Rye	1.00 ± 0.53 (a)
2023	Soy	1.44 ± 0.63 (a)
2024	Corn	2.42 ± 1.64 (a)
2024	Sorghum	1.61 ± 1.07 (a)
2024	Sorghum + Rye	0.87 ± 0.54 (a)
2024	Soy	1.35 ± 1.36 (a)
2023-2024	Corn	2.12 ± 1.14 (a)
2023-2024	Sorghum	2.84 ± 1.71 (a)
2023-2024	Sorghum + Rye	1.86 ± 0.92 (a)
2023-2024	Soy	1.39 ± 1.03 (a)

Values are mean ± SD. Letters indicate Tukey-adjusted groups within year/period (=0.05). Costs: N O (\$16.18/kg N) + Leaching (\$18.54/kg N). Global ANOVA Treatment effect for Cost: p = 0.004.

```
library(dplyr)
library(tidyr)
library(gt)
library(emmeans)
library(multcomp)

# --- 1. PREPARE PLOT-LEVEL DATA (Averaging Row/Interrow) ---

# N2O: Average Row/Interrow daily, THEN sum for the year
n2o_plot <- interpolated_flux_data |>
  group_by(plot, treatment, year, date) |>
  summarise(daily_plot_flux = mean(interpolated_flux, na.rm = TRUE), .groups = "drop") |>
  group_by(plot, treatment, year) |>
  summarise(
    n2o_kg_ha = sum(daily_plot_flux, na.rm = TRUE) / 1000,
    .groups = "drop"
  )

# Leaching: Ensure Plot Level
leach_plot <- annual_leaching |>
  select(year, plot, treatment, leaching_kg_ha = annual_cumulative_leaching)

# Join & Calculate Plot EDC
plot_economics <- n2o_plot |>
  inner_join(leach_plot, by = c("year", "plot", "treatment")) |>
  mutate(edc_dollars = (n2o_kg_ha * 16.18) + (leaching_kg_ha * 18.54))
```

```

# --- 2. CALCULATE 2-YEAR TOTALS (Method of Moments) ---

# A. Continuous Systems (Real Sums)
continuous_2year <- plot_economics |>
  filter(year %in% c(2023, 2024), treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  group_by(plot, treatment) |>
  summarise(
    n2o_kg_ha = sum(n2o_kg_ha, na.rm = TRUE),
    leaching_kg_ha = sum(leaching_kg_ha, na.rm = TRUE),
    edc_dollars = sum(edc_dollars, na.rm = TRUE),
    count = n(),
    .groups = "drop"
  ) |>
  filter(count == 2) |>
  select(-count)

# B. Rotation Systems (Variance Propagation)
# Annual stats (using all plots)
rotation_annual_stats <- plot_economics |>
  filter(year %in% c(2023, 2024), treatment %in% c("Corn", "Soy")) |>
  group_by(treatment, year) |>
  summarise(
    mu_n2o = mean(n2o_kg_ha, na.rm = TRUE), var_n2o = var(n2o_kg_ha, na.rm = TRUE),
    mu_leach = mean(leaching_kg_ha, na.rm = TRUE), var_leach = var(leaching_kg_ha, na.rm = TRUE),
    mu_edc = mean(edc_dollars, na.rm = TRUE), var_edc = var(edc_dollars, na.rm = TRUE),
    .groups = "drop"
  )

# Propagate to 2-Year Totals
rotation_2year_stats <- rotation_annual_stats |>
  group_by(treatment) |>
  summarise(
    final_mu_n2o = sum(mu_n2o), final_sd_n2o = sqrt(sum(var_n2o)),
    final_mu_leach = sum(mu_leach), final_sd_leach = sqrt(sum(var_leach)),
    final_mu_edc = sum(mu_edc), final_sd_edc = sqrt(sum(var_edc)),
    .groups = "drop"
  )

# Synthetic Plots for ANOVA (n=3)
rotation_synthetic_plots <- rotation_2year_stats |>
  group_by(treatment) |>
  group_modify(~ {
    n_sim <- 3
    make_dist <- function(n, mu, sd) {
      x <- rnorm(n)
      if(sd > 0) x_final <- ((x - mean(x)) / sd(x)) * sd + mu else x_final <- rep(mu, n)
      return(pmax(0, x_final))
    }
    tibble(
      plot = paste0("Syn_", 1:n_sim),
      n2o_kg_ha = make_dist(n_sim, .x$final_mu_n2o, .x$final_sd_n2o),
      leaching_kg_ha = make_dist(n_sim, .x$final_mu_leach, .x$final_sd_leach),
      edc_dollars = make_dist(n_sim, .x$final_mu_edc, .x$final_sd_edc)
    )
  })

```

```

    )
  }) |>
  ungroup()

# Combine Real + Synthetic
final_2year_data <- bind_rows(continuous_2year, rotation_synthetic_plots) |>
  mutate(year = "2023-2024")

# Master Dataset
stats_input_data <- bind_rows(
  plot_economics |> mutate(year = as.character(year)),
  final_2year_data
)

# --- 3. STATISTICAL LOOP (3 SIG FIGS) ---

calc_stat_column <- function(data, col_name, is_money = FALSE) {
  # Fit Model
  fit <- lm(as.formula(paste(col_name, "~ treatment")), data = data)

  # Get Letters
  lets <- tryCatch({
    emmeans(fit, ~ treatment) |>
      cld(adjust = "tukey", Letters = letters, alpha = 0.05) |>
      as_tibble() |>
      mutate(letter = trimws(.group)) |>
      select(treatment, letter)
  }, error = function(e) tibble(treatment = unique(data$treatment), letter = ""))

  # Get Summary Stats & Format to 3 Sig Figs
  data |>
    group_by(treatment) |>
    summarise(mean = mean(.data[[col_name]], na.rm = TRUE), sd = sd(.data[[col_name]], na.rm = TRUE), .)
    left_join(lets, by = "treatment") |>
    mutate(
      # Formatting Logic: 3 Significant Figures
      mean_str = formatC(signif(mean, 3), digits = 3, format = "fg", flag = "#"),
      sd_str = formatC(signif(sd, 3), digits = 3, format = "fg", flag = "#"),
      # Remove trailing dots (e.g., "123." -> "123")
      mean_str = sub("\\.$", "", mean_str),
      sd_str = sub("\\.$", "", sd_str),

      result_str = if(is_money) {
        sprintf("$$s \u00B1 %s (%s)", mean_str, sd_str, letter)
      } else {
        sprintf("%s \u00B1 %s (%s)", mean_str, sd_str, letter)
      }
    ) |>
    select(treatment, result_str)
}

# Run Loop
table_body <- stats_input_data |>

```

```

group_by(year) |>
group_modify(~ {
  n2o <- calc_stat_column(.x, "n2o_kg_ha")
  leach <- calc_stat_column(.x, "leaching_kg_ha")
  edc <- calc_stat_column(.x, "edc_dollars", is_money = TRUE)
  n2o |> rename(N20 = result_str) |>
  left_join(leach |> rename(Leach = result_str), by = "treatment") |>
  left_join(edc |> rename(Cost = result_str), by = "treatment")
}) |>
ungroup()

# --- 4. SOURCE OF VARIATION ---
# (Using plot_economics for correct global stats)
aov_n2o <- summary(aov(n2o_kg_ha ~ as.factor(year) * treatment, data = plot_economics))[[1]]
aov_leach <- summary(aov(leaching_kg_ha ~ as.factor(year) * treatment, data = plot_economics))[[1]]
aov_edc <- summary(aov(edc_dollars ~ as.factor(year) * treatment, data = plot_economics))[[1]]

format_p <- function(p) ifelse(p < 0.001, "< 0.001", sprintf("%.3f", p))

source_var <- tibble(
  year = "Source of Variation",
  treatment = c("Treatment", "Year", "Year × Treatment"),
  N20 = c(format_p(aov_n2o[2, "Pr(>F)"]), format_p(aov_n2o[1, "Pr(>F)"]), format_p(aov_n2o[3, "Pr(>F)"])),
  Leach = c(format_p(aov_leach[2, "Pr(>F)"]), format_p(aov_leach[1, "Pr(>F)"]), format_p(aov_leach[3, "Pr(>F)"])),
  Cost = c(format_p(aov_edc[2, "Pr(>F)"]), format_p(aov_edc[1, "Pr(>F)"]), format_p(aov_edc[3, "Pr(>F)"])),
)

# --- 5. FINALIZE TABLE & RENDER ---
trt_order <- c("Corn", "Soy", "Sorghum", "Sorghum + Rye")

final_table <- bind_rows(table_body, source_var) |>
mutate(
  order_helper = case_when(
    year == "2023" ~ 1, year == "2024" ~ 2,
    year == "2023-2024" ~ 3, year == "Source of Variation" ~ 4
  ),
  treatment_factor = factor(treatment, levels = c(trt_order, "Treatment", "Year", "Year × Treatment"))
) |>
arrange(order_helper, treatment_factor) |>
select(Year = year, Treatment = treatment, `N20 Loss` = N20, `N Leaching` = Leach, `Env. Damage Cost` = Cost)

ft <- final_table |>
gt() |>
tab_header(title = md("**Table 3. Annual nitrogen losses and environmental costs**")) |>
cols_label(
  `N20 Loss` = md("N<sub>2</sub>O Loss<br>(kg N ha<sup>-1</sup>)"),
  `N Leaching` = md("N Leaching<br>(kg N ha<sup>-1</sup>)"),
  `Env. Damage Cost` = md("Total Environmental Cost<br>(USD ha<sup>-1</sup>)")
) |>
cols_align(align = "center", columns = -Treatment) |>
tab_style(
  style = cell_borders(sides = "top", color = "gray", weight = px(2)),
  locations = cells_body(rows = Year == "Source of Variation")
)

```

Table 3. Annual nitrogen losses and environmental costs

Year	Treatment	N2O Loss (kg N ha ⁻¹)
2023	Corn	1.89 ± 0.814 (a)
2023	Soy	1.45 ± 0.628 (a)
2023	Sorghum	1.23 ± 0.685 (a)
2023	Sorghum + Rye	0.996 ± 0.526 (a)
2024	Corn	2.42 ± 1.64 (a)
2024	Soy	1.35 ± 1.36 (a)
2024	Sorghum	1.61 ± 1.07 (a)
2024	Sorghum + Rye	0.868 ± 0.544 (a)
2023-2024	Corn	4.31 ± 1.83 (a)
2023-2024	Soy	2.80 ± 1.50 (a)
2023-2024	Sorghum	2.84 ± 1.71 (a)
2023-2024	Sorghum + Rye	1.86 ± 0.920 (a)
Source of Variation	Treatment	0.151
Source of Variation	Year	0.739
Source of Variation	Year × Treatment	0.883

Values are mean ± SD (3 significant figures). Letters indicate Tukey-adjusted groups within year/period (=0.05). Costs: N O (\$16.18/kg N) + Leaching (\$18.54/kg N). Two-year totals for Corn/Soy estimated using variance propagation of annual crop phases.

```

) |>
tab_style(
  style = cell_text(weight = "bold"),
  locations = cells_body(rows = Year == "Source of Variation")
) |>
tab_footnote(
  footnote = "Values are mean ± SD (3 significant figures). Letters indicate Tukey-adjusted groups wi
)
ft

# Save as RTF

gtsave(ft, "tables/table3_nitrogen_losses_environmental_costs.rtf")

library(car) # For Levene's Test

# --- FUNCTION: CHECK ASSUMPTIONS ---
check_anova_assumptions <- function(data, col_name, label) {
  # 1. Fit the Global Model
  # We use the same model structure as your Table 3 footer
  model <- aov(as.formula(paste(col_name, "~ year * treatment")), data = data)

  # 2. Normality Test (Shapiro-Wilk on Residuals)
  # H0: Residuals are Normal. (We WANT p > 0.05)

```

```

shapiro <- shapiro.test(residuals(model))

# 3. Homogeneity of Variance (Levene's Test)
# H0: Variances are Equal. (We WANT p > 0.05)
# Note: using 'as.factor(year)' to treat year as a group
levene <- leveneTest(as.formula(paste(col_name, "~ as.factor(year) * treatment")), data = data)

# 4. Return Summary
tibble(
  Variable = label,
  `Shapiro-Wilk P` = sprintf("%.3f", shapiro$p.value),
  `Normality Verdict` = ifelse(shapiro$p.value > 0.05, "PASS", "FAIL (Transform?)"),
  `Levene P` = sprintf("%.3f", levene$`Pr(>F)`[1]),
  `Homogeneity Verdict` = ifelse(levene$`Pr(>F)`[1] > 0.05, "PASS", "FAIL (Weighted LS?)")
)
}

# --- RUN CHECKS ON ALL 3 VARIABLES ---
# We use 'plot_economics' because that is the real data used for the global ANOVA
assumptions_table <- bind_rows(
  check_anova_assumptions(plot_economics, "n2o_kg_ha", "N2O Loss"),
  check_anova_assumptions(plot_economics, "leaching_kg_ha", "N Leaching"),
  check_anova_assumptions(plot_economics, "edc_dollars", "Env. Damage Cost")
)

# Print the Table
print(assumptions_table)

# A tibble: 3 x 5
  Variable `Shapiro-Wilk P` `Normality Verdict` `Levene P` `Homogeneity Verdict`
  <chr>    <chr>            <chr>          <chr>      <chr>
1 N2O Loss 0.198          PASS           0.866      PASS
2 N Leach~ 0.001          FAIL (Transform?) 0.073      PASS
3 Env. Da~ 0.003          FAIL (Transform?) 0.123      PASS

library(dplyr)
library(tidyr)
library(gt)
library(emmeans)
library(multcomp)

# --- 1. PREPARE PLOT-LEVEL DATA (With Row/Interrow Averaging) ---
n2o_plot <- interpolated_flux_data |>
  group_by(plot, treatment, year, date) |>
  summarise(daily_plot_flux = mean(interpolated_flux, na.rm = TRUE), .groups = "drop") |>
  group_by(plot, treatment, year) |>
  summarise(n2o_kg_ha = sum(daily_plot_flux, na.rm = TRUE) / 1000, .groups = "drop")

leach_plot <- annual_leaching |>
  select(year, plot, treatment, leaching_kg_ha = annual_cumulative_leaching)

plot_economics <- n2o_plot |>
  inner_join(leach_plot, by = c("year", "plot", "treatment")) |>
  mutate(edc_dollars = (n2o_kg_ha * 16.18) + (leaching_kg_ha * 18.54))

```

```

# --- 2. CALCULATE 2-YEAR TOTALS (Method of Moments) ---
continuous_2year <- plot_economics |>
  filter(year %in% c(2023, 2024), treatment %in% c("Sorghum", "Sorghum + Rye")) |>
  group_by(plot, treatment) |>
  summarise(
    n2o_kg_ha = sum(n2o_kg_ha, na.rm = TRUE),
    leaching_kg_ha = sum(leaching_kg_ha, na.rm = TRUE),
    edc_dollars = sum(edc_dollars, na.rm = TRUE),
    count = n(), .groups = "drop"
  ) |> filter(count == 2) |> select(-count)

rotation_annual_stats <- plot_economics |>
  filter(year %in% c(2023, 2024), treatment %in% c("Corn", "Soy")) |>
  group_by(treatment, year) |>
  summarise(
    mu_n2o = mean(n2o_kg_ha, na.rm = TRUE), var_n2o = var(n2o_kg_ha, na.rm = TRUE),
    mu_leach = mean(leaching_kg_ha, na.rm = TRUE), var_leach = var(leaching_kg_ha, na.rm = TRUE),
    mu_edc = mean(edc_dollars, na.rm = TRUE), var_edc = var(edc_dollars, na.rm = TRUE),
    .groups = "drop"
  )

rotation_2year_stats <- rotation_annual_stats |>
  group_by(treatment) |>
  summarise(
    final_mu_n2o = sum(mu_n2o), final_sd_n2o = sqrt(sum(var_n2o)),
    final_mu_leach = sum(mu_leach), final_sd_leach = sqrt(sum(var_leach)),
    final_mu_edc = sum(mu_edc), final_sd_edc = sqrt(sum(var_edc)), .groups = "drop"
  )

rotation_synthetic_plots <- rotation_2year_stats |>
  group_by(treatment) |>
  group_modify(~ {
    n_sim <- 3
    make_dist <- function(n, mu, sd) {
      x <- rnorm(n)
      if(sd > 0) x_final <- ((x - mean(x)) / sd(x)) * sd + mu else x_final <- rep(mu, n)
      return(pmax(0, x_final))
    }
    tibble(
      plot = paste0("Syn_", 1:n_sim),
      n2o_kg_ha = make_dist(n_sim, .x$final_mu_n2o, .x$final_sd_n2o),
      leaching_kg_ha = make_dist(n_sim, .x$final_mu_leach, .x$final_sd_leach),
      edc_dollars = make_dist(n_sim, .x$final_mu_edc, .x$final_sd_edc)
    )
  }) |> ungroup()

final_2year_data <- bind_rows(continuous_2year, rotation_synthetic_plots) |> mutate(year = "2023-2024")

stats_input_data <- bind_rows(plot_economics |> mutate(year = as.character(year)), final_2year_data)

# --- 3. STATISTICAL LOOP (TRANSFORMED STATS, REAL MEANS) ---

calc_stat_column <- function(data, col_name, is_money = FALSE) {

```

```

# DECISION: Log-transform for Leaching and Cost, Raw for N2O
if (col_name %in% c("leaching_kg_ha", "edc_dollars")) {
  f <- as.formula(paste("log10(", col_name, " + 1) ~ treatment"))
} else {
  f <- as.formula(paste(col_name, "~ treatment"))
}

fit <- lm(f, data = data)

# Get Letters (based on the TRANSFORMED model)
lets <- tryCatch({
  emmeans(fit, ~ treatment) |>
    cld(adjust = "tukey", Letters = letters, alpha = 0.05) |>
    as_tibble() |>
    mutate(letter = trimws(.group)) |>
    select(treatment, letter)
}, error = function(e) tibble(treatment = unique(data$treatment), letter = ""))

# Get Summary Stats (Arithmetic Mean/SD for display)
data |>
  group_by(treatment) |>
  summarise(mean = mean(.data[[col_name]], na.rm = TRUE), sd = sd(.data[[col_name]], na.rm = TRUE), .)
  left_join(lets, by = "treatment") |>
  mutate(
    mean_str = formatC(signif(mean, 3), digits = 3, format = "fg", flag = "#"),
    sd_str = formatC(signif(sd, 3), digits = 3, format = "fg", flag = "#"),
    mean_str = sub("\\.\\$", "", mean_str),
    sd_str = sub("\\.\\$", "", sd_str),
    result_str = if(is_money) sprintf("$%s \u00B1 %s (%s)", mean_str, sd_str, letter) else sprintf("%s", mean_str, sd_str, letter)
  ) |>
  select(treatment, result_str)
}

table_body <- stats_input_data |>
  group_by(year) |>
  group_modify(~ {
    n2o <- calc_stat_column(.x, "n2o_kg_ha")
    leach <- calc_stat_column(.x, "leaching_kg_ha")
    edc <- calc_stat_column(.x, "edc_dollars", is_money = TRUE)
    n2o |> rename(N2O = result_str) |>
      left_join(leach |> rename(Leach = result_str), by = "treatment") |>
      left_join(edc |> rename(Cost = result_str), by = "treatment")
  }) |> ungroup()

# --- 4. SOURCE OF VARIATION (Using Log Models where needed) ---
aov_n2o <- summary(aov(n2o_kg_ha ~ as.factor(year) * treatment, data = plot_economics))[[1]]
aov_leach <- summary(aov(log10(leaching_kg_ha+1) ~ as.factor(year) * treatment, data = plot_economics))[[1]]
aov_edc <- summary(aov(log10(edc_dollars+1) ~ as.factor(year) * treatment, data = plot_economics))[[1]]

format_p <- function(p) ifelse(p < 0.001, "< 0.001", sprintf("%.3f", p))

source_var <- tibble(
  year = "Source of Variation",

```

Table 3. Annual nitrogen losses and environmental costs

Year	Treatment	N2O Loss (kg N ha ⁻¹)
2023	Corn	1.89 ± 0.814 (a)
2023	Soy	1.45 ± 0.628 (a)
2023	Sorghum	1.23 ± 0.685 (a)
2023	Sorghum + Rye	0.996 ± 0.526 (a)
2024	Corn	2.42 ± 1.64 (a)
2024	Soy	1.35 ± 1.36 (a)
2024	Sorghum	1.61 ± 1.07 (a)
2024	Sorghum + Rye	0.868 ± 0.544 (a)
2023-2024	Corn	4.31 ± 1.83 (a)
2023-2024	Soy	2.80 ± 1.50 (a)
2023-2024	Sorghum	2.84 ± 1.71 (a)
2023-2024	Sorghum + Rye	1.86 ± 0.920 (a)
Source of Variation	Treatment	0.151
Source of Variation	Year	0.739
Source of Variation	Year × Treatment	0.883

Values are arithmetic mean ± SD (3 sig figs). Letters indicate Tukey-adjusted groups derived from log-transformed data for leaching/costs and non-transformed data for N2O ($\alpha=0.05$).

```

treatment = c("Treatment", "Year", "Year × Treatment"),
N2O = c(format_p(aov_n2o[2, "Pr(>F)"]), format_p(aov_n2o[1, "Pr(>F)"]), format_p(aov_n2o[3, "Pr(>F)"]),
Leach = c(format_p(aov_leach[2, "Pr(>F)"]), format_p(aov_leach[1, "Pr(>F)"]), format_p(aov_leach[3, "Pr(>F)"]),
Cost = c(format_p(aov_edc[2, "Pr(>F)"]), format_p(aov_edc[1, "Pr(>F)"]), format_p(aov_edc[3, "Pr(>F)"]),
)

# --- 5. FINALIZE ---
trt_order <- c("Corn", "Soy", "Sorghum", "Sorghum + Rye")
final_table <- bind_rows(table_body, source_var) |>
  mutate(
    order_helper = case_when(year == "2023" ~ 1, year == "2024" ~ 2, year == "2023-2024" ~ 3, year == "Source of Variation" ~ 4),
    treatment_factor = factor(treatment, levels = c(trt_order, "Treatment", "Year", "Year × Treatment"))
  ) |>
  arrange(order_helper, treatment_factor) |>
  select(Year = year, Treatment = treatment, `N2O Loss` = N2O, `N Leaching` = Leach, `Env. Damage Cost` = Cost)

final_table |> gt() |>
  tab_header(title = md("**Table 3. Annual nitrogen losses and environmental costs**")) |>
  cols_label(`N2O Loss` = md("N<sub>2</sub>O Loss<br>(kg N ha<sup>-1</sup>)), `N Leaching` = md("N Leaching"), `Env. Damage Cost` = md("Env. Damage Cost")) |>
  cols_align(aligned = "center", columns = -Treatment) |>
  tab_style(style = cell_borders(sides = "top", color = "gray", weight = px(2)), locations = cells_body(rows = 1)) |>
  tab_style(style = cell_text(weight = "bold"), locations = cells_body(rows = 4)) |>
  tab_footnote(footnote = "Values are arithmetic mean ± SD (3 sig figs). Letters indicate Tukey-adjusted groups")

```

```

# --- QUICK P-VALUE CHECKER (LEACHING) ---

# 1. Fit the Log-Transformed Model (Same as your Table 3)
fit_leach <- lm(log10(leaching_kg_ha + 1) ~ year * treatment, data = plot_economics)

# 2. Extract the Specific Contrasts (Sorghum vs. Sorghum + Rye)
library(emmeans)
contrasts <- emmeans(fit_leach, ~ treatment | year) |>
  contrast(method = "pairwise") |>
  as_tibble() |>
  filter(contrast == "Sorghum - (Sorghum + Rye)") |>
  select(year, contrast, p.value)

# 3. Print the Result
print(contrasts)

# A tibble: 2 x 3
   year contrast                                p.value
  <dbl> <fct>                                <dbl>
1  2023 Sorghum - (Sorghum + Rye)          0.454
2  2024 Sorghum - (Sorghum + Rye)          0.269

library(tidyverse)
library(readxl)
library(patchwork) # For combining panels

# --- 1. LOAD & CLEAN TEXTURE DATA (Your Code) ---
# Note: Ensure the path to the Excel file is correct in your local environment
texture_raw <- read_excel("data/PSA Report - AGRON-McDaniel-Potter-2024.xlsm",
  sheet = "Main Data File format", n_max = 22
)

# Ensure leaching data is available (re-load if necessary)
if (!exists("leaching_data")) {
  leaching_data <- read_csv("data/clean_leaching.csv") |>
    janitor::clean_names() |>
    mutate(
      date = as.Date(date),
      year = year(date)
    ) |>
    filter(year %in% c(2023, 2024))
}

# Calculate annual cumulative nitrogen leaching per plot
plot_area_ft2 <- 120 * 160
plot_area_ha <- plot_area_ft2 / 107639.1041671

annual_leaching <- leaching_data |>
  mutate(plot = as.numeric(plot)) |>
  group_by(plot, treatment, year) |>
  summarise(
    # mg -> kg, then divide by hectares
    annual_cumulative_leaching = (max(cumulative_n_loss_mg, na.rm = TRUE) / 1e6) / plot_area_ha,
    .groups = "drop",

```

```

    annual_drainage_l = max(cumulative_flow_l, na.rm = TRUE),
    annual_drainage_mm = (annual_drainage_l / plot_area_ha) / 10000
  ) |>
  filter(is.finite(annual_cumulative_leaching), annual_cumulative_leaching >= 0) |>
  arrange(desc(annual_cumulative_leaching))

# render whole table in .html
annual_leaching

```

```

# A tibble: 30 x 6
  plot treatment year annual_cumulative_leaching annual_drainage_l
  <dbl> <chr>      <dbl>                <dbl>                <dbl>
1    11 Soy      2024                 32.7                 513548.
2    14 Soy      2024                 17.3                 677785.
3     1 Soy      2024                 12.6                 263112.
4    14 Corn     2023                 10.7                 431506.
5    15 Sorghum  2024                 9.68                 295652.
6    11 Corn     2023                 9.04                 204333.
7     4 Corn     2024                 7.80                 114603.
8     6 Corn     2024                 7.51                  73831.
9    15 Sorghum  2023                 7.19                 183570.
10    8 Corn     2023                 6.33                 119028.
# i 20 more rows
# i 1 more variable: annual_drainage_mm <dbl>

```

```
gt(annual_leaching)
```

```

texture <- texture_raw %>%
  select(Sample, Sand = S, Silt = T, Clay = C) %>%
  separate(Sample, into = c("ID", "Plot"), sep = "_") %>%
  mutate(Plot = as.numeric(Plot)) %>%
  drop_na() %>%
  mutate(
    Sand = as.numeric(Sand),
    Silt = as.numeric(Silt),
    Clay = as.numeric(Clay)
  ) %>%
  select(Plot, Sand, Silt, Clay) # We only need Plot and Texture cols

# --- 2. JOIN WITH DRAINAGE & LEACHING DATA ---
# We use 'annual_leaching' from your previous analysis steps
# Ensure 'drainage_mm' is the correct column name for water volume in your dataset
plot_hydrology <- annual_leaching %>%
  select(year, plot, treatment, drainage_mm = annual_drainage_mm,
    leaching_kg_ha = annual_cumulative_leaching) %>%
  left_join(texture, by = c("plot" = "Plot")) %>%
  mutate(
    year = as.factor(year),
    # Rename Corn → Maize, Soy → Soybean for publication
    Treatment = case_when(
      treatment == "Corn" ~ "Maize",
      treatment == "Soy" ~ "Soybean",
      treatment == "Sorghum + Rye" ~ "Sorghum + WCC",
      TRUE ~ treatment
    )
  )

```

plot	treatment	year	annual_cumulative_leaching	annual_drainage_l	annual_drainage_mm
11	Soy	2024	32.73067	513548	287.90
14	Soy	2024	17.27574	677785	379.98
1	Soy	2024	12.62555	263112	147.50
14	Corn	2023	10.70957	431506	241.91
15	Sorghum	2024	9.68307	295652	165.74
11	Corn	2023	9.04221	204333	114.55
4	Corn	2024	7.79739	114603	64.24
6	Corn	2024	7.50981	73831	41.39
15	Sorghum	2023	7.19259	183570	102.91
8	Corn	2023	6.32722	119028	66.73
4	Soy	2023	6.17153	98727	55.34
8	Soy	2024	3.82359	123090	69.00
2	Sorghum	2023	3.80110	64515	36.16
1	Corn	2023	2.94363	60249	33.77
2	Sorghum	2024	2.90622	120380	67.48
3	Sorghum	2024	2.10420	55744	31.25
10	Corn	2024	1.88285	33156	18.58
12	Sorghum + Rye	2024	0.93288	419223	235.02
12	Sorghum + Rye	2023	0.52630	152579	85.53
7	Sorghum + Rye	2024	0.46271	98652	55.30
10	Soy	2023	0.38911	12791	7.17
13	Sorghum + Rye	2024	0.24386	26695	14.96
5	Sorghum	2024	0.24144	5705	3.19
13	Sorghum + Rye	2023	0.23301	4035	2.26
7	Sorghum + Rye	2023	0.17656	60695	34.02
5	Sorghum	2023	0.14099	1325	0.74
6	Soy	2023	0.07921	1192	0.66
3	Sorghum	2023	0.05825	840	0.47
9	Sorghum + Rye	2024	0.00601	1999	1.12
9	Sorghum + Rye	2023	0.00414	1060	0.59

```

    ),
    Treatment = factor(Treatment,
                        levels = c("Maize", "Soybean", "Sorghum", "Sorghum + WCC"))
  )

# Subtle colors for Maize/Soybean, standard for Sorghum treatments
supp_colors <- c("Maize"      = "#B0B0B0", # gray
                 "Soybean"    = "#A8C8E8", # light blue
                 "Sorghum"     = "#D8D97AFF",
                 "Sorghum + WCC" = "#95C36EFF")

supp_shapes <- c("Maize" = 22, "Soybean" = 24,
                 "Sorghum" = 21, "Sorghum + WCC" = 23)

# Helper: extract per-treatment lm equation + R2 annotation labels
lm_labels <- function(df, xvar, yvar) {
  df |>
    group_by(Treatment) |>
    summarise(
      lbl = {
        mod <- lm(get(yvar) ~ get(xvar))
        b0 <- round(coef(mod)[1], 1)
        b1 <- round(coef(mod)[2], 2)
        r2 <- round(summary(mod)$r.squared, 2)
        paste0("y = ", b1, "x + ", b0, " R\u00b2 = ", r2)
      },
      x_pos = max(get(xvar), na.rm = TRUE),
      y_pos = max(get(yvar), na.rm = TRUE),
      .groups = "drop"
    )
}

# --- 3. GENERATE PLOTS ---

# PANEL (a): Drainage vs. N Leaching Load (swapped from old Panel B)
eq_loss <- lm_labels(plot_hydrology, "drainage_mm", "leaching_kg_ha")

p_loss <- ggplot(plot_hydrology, aes(x = drainage_mm, y = leaching_kg_ha)) +
  # Per-treatment regression lines
  geom_smooth(aes(color = Treatment), method = "lm", se = FALSE, linewidth = 0.7) +
  # Points with shape differences
  geom_point(aes(fill = Treatment, shape = Treatment), size = 3.5, alpha = 0.9,
             color = "black", stroke = 0.4) +
  scale_fill_manual(values = supp_colors) +
  scale_color_manual(values = supp_colors) +
  scale_shape_manual(values = supp_shapes) +
  # Annotate per-treatment equations
  geom_text(data = eq_loss, aes(x = -Inf, y = Inf, label = lbl, color = Treatment),
            hjust = -0.05, vjust = seq(1.5, by = 1.3, length.out = nrow(eq_loss)),
            size = 2.8, show.legend = FALSE, fontface = "italic") +
  labs(tag = "(a)", x = "Annual Drainage (mm)",
       y = expression("Cumulative Leached N (kg N ha"^-1*")"),
       fill = "Treatment", shape = "Treatment", color = "Treatment") +

```

```

theme_publication(base_size = 10) +
theme(legend.position = "top",
      plot.tag = element_text(face = "bold", size = 11),
      plot.tag.position = "topleft")

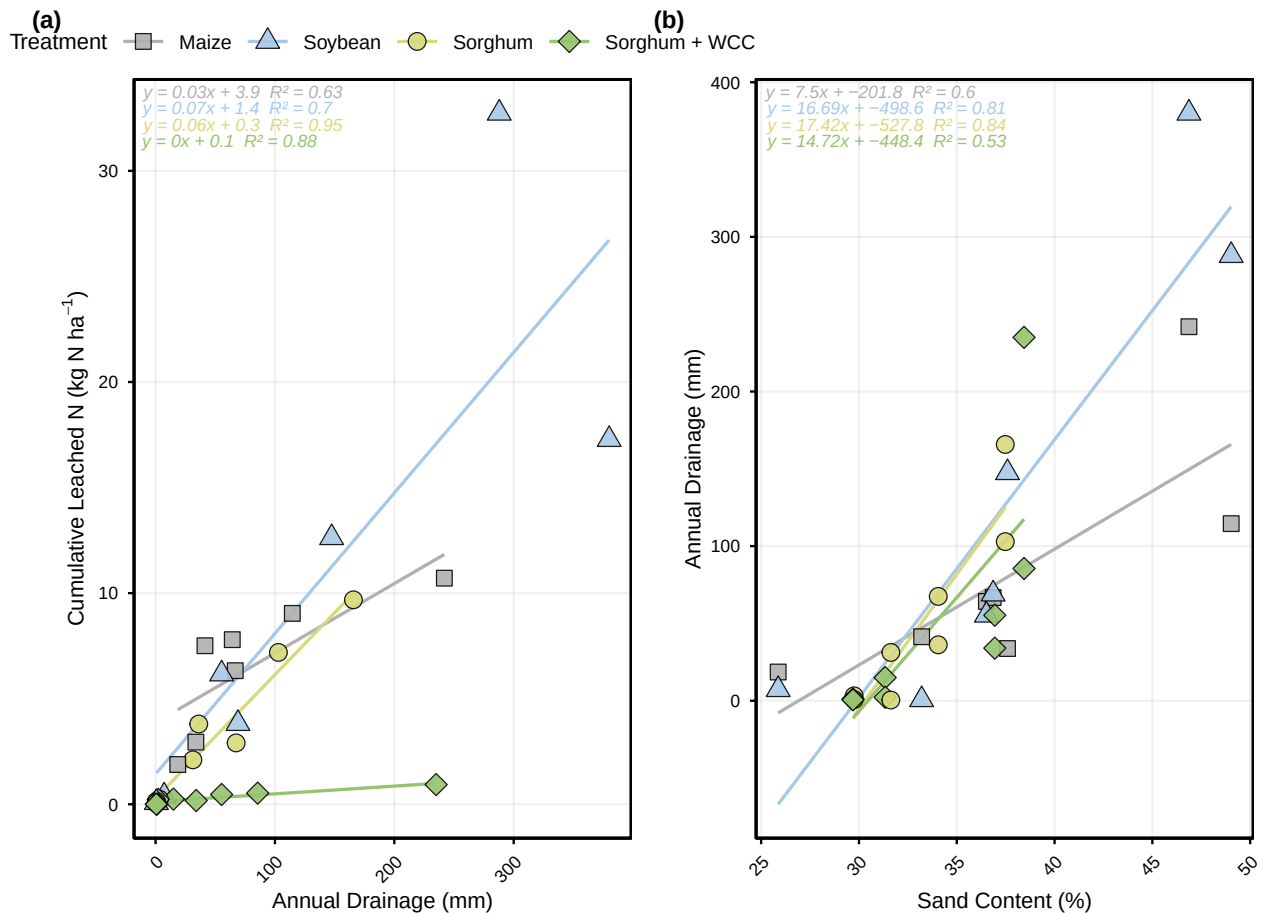
# PANEL (b): Texture vs. Drainage (swapped from old Panel A)
eq_tex <- lm_labels(plot_hydrology, "Sand", "drainage_mm")

p_texture <- ggplot(plot_hydrology, aes(x = Sand, y = drainage_mm)) +
  geom_smooth(aes(color = Treatment), method = "lm", se = FALSE, linewidth = 0.7) +
  geom_point(aes(fill = Treatment, shape = Treatment), size = 3.5, alpha = 0.9,
            color = "black", stroke = 0.4) +
  scale_fill_manual(values = supp_colors) +
  scale_color_manual(values = supp_colors) +
  scale_shape_manual(values = supp_shapes) +
  geom_text(data = eq_tex, aes(x = -Inf, y = Inf, label = lbl, color = Treatment),
            hjust = -0.05, vjust = seq(1.5, by = 1.3, length.out = nrow(eq_tex)),
            size = 2.8, show.legend = FALSE, fontface = "italic") +
  labs(tag = "(b)", x = "Sand Content (%)",
       y = "Annual Drainage (mm)") +
  theme_publication(base_size = 10) +
  theme(legend.position = "none",
        plot.tag = element_text(face = "bold", size = 11),
        plot.tag.position = "topleft")

# --- 4. COMBINE & SAVE ---
figure_s3 <- p_loss + p_texture + plot_layout(widths = c(1, 1))

print(figure_s3)

```



```
ggsave("FiguresS_Hydrology.tiff", figure_s3, width = 174, height = 90,
       units = "mm", dpi = 600, compression = "lzw")
```

```
library(tidyverse)

# 1. DEFINE CONSTANTS
plot_area_ft2 <- 120 * 160
plot_area_ha <- plot_area_ft2 / 107639.1041671

# 2. PREPARE DATA
# We use the raw 'leaching_data' (daily/weekly measurements)
hydrograph_data <- leaching_data %>%
  arrange(plot, date) %>%
  group_by(plot, year) %>% # Reset calculation for each year/plot combo
  mutate(
    # Convert Cumulative Liters -> mm Depth
    cumulative_mm = (cumulative_flow_l / plot_area_ha) / 10000,

    # Optional: If your data doesn't reset to 0 at Jan 1, create a "Yearly Cumulative"
    # This ensures every line starts at 0 for the graph
    cumulative_mm_reset = cumulative_mm - min(cumulative_mm, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  mutate(plot = as.factor(plot))
```

```

# 3. PLOT CUMULATIVE HYDROGRAPH
p_hydro <- ggplot(hydrograph_data, aes(x = date, y = cumulative_mm_reset, group = plot, color = treatment))
# Draw lines for each plot
geom_line(alpha = 0.8, size = 0.7) +

# Split by Year to see seasonal differences clearly
facet_wrap(~year, scales = "free_x") +

# Rename treatments for display
aes(color = case_when(
  treatment == "Corn"           ~ "Maize",
  treatment == "Soy"            ~ "Soybean",
  treatment == "Sorghum + Rye" ~ "Sorghum + WCC",
  TRUE                          ~ treatment
)) +

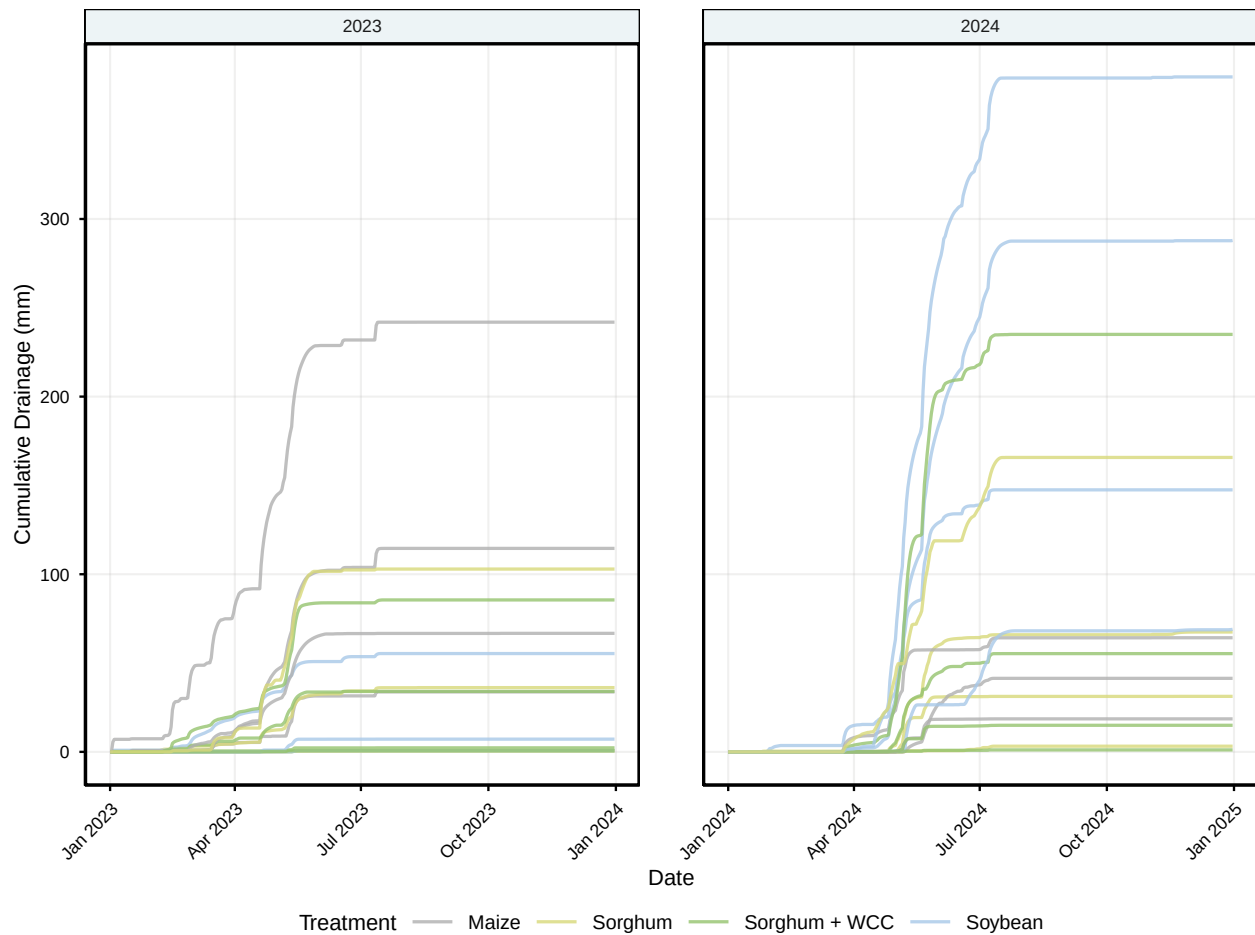
# Colors - subtle Maize/Soybean
scale_color_manual(values = c("Maize"           = "#B0B0B0",
                              "Soybean"         = "#A8C8E8",
                              "Sorghum"         = "#D8D97AFF",
                              "Sorghum + WCC"    = "#95C36EFF")) +

# Labels - no title per McDaniel
labs(x = "Date",
     y = "Cumulative Drainage (mm)",
     color = "Treatment") +

# Theme
theme_publication(base_size = 10) +
theme(
  legend.position = "bottom"
)

print(p_hydro)

```



```
library(broom) # For nice summary tables

# --- TEST 1: The "Texture Driver" (For Panel A) ---
# Question: Does Sand % significantly predict Drainage?
# We use a Linear Model (lm).
tex_stats <- lm(drainage_mm ~ Sand, data = plot_hydrology)

print("--- TEXTURE VS DRAINAGE STATISTICS ---")
```

```
[1] "--- TEXTURE VS DRAINAGE STATISTICS ---"
```

```
print(anova(tex_stats))
```

Analysis of Variance Table

```
Response: drainage_mm
      Df Sum Sq Mean Sq F value Pr(>F)
Sand    1 174714  174714    53.8 5.5e-08 ***
Residuals 28  90999    3250
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(summary(tex_stats))
```

Call:

```
lm(formula = drainage_mm ~ Sand, data = plot_hydrology)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-135.17	-26.98	-3.94	17.27	157.92

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-378.08	63.18	-5.98	1.9e-06 ***
Sand	12.80	1.75	7.33	5.5e-08 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 57 on 28 degrees of freedom

Multiple R-squared: 0.658, Adjusted R-squared: 0.645

F-statistic: 53.8 on 1 and 28 DF, p-value: 5.53e-08

Look for:

1. Multiple R-squared (The strength of the link)

2. P-value for 'Sand' (Is it significant?)

--- **TEST** 2: The "Uncoupling" Effect (For Panel B) ---

Question: Is the slope of Leaching vs. Drainage different for WCC?

We use ANCOVA. The key is the INTERACTION term (*).

Model: Leaching ~ Drainage + Treatment + (Drainage * Treatment)

First, set "Sorghum + Rye" as the reference group to see if others differ from IT

```
plot_hydrology$treatment <- relevel(factor(plot_hydrology$treatment), ref = "Sorghum + Rye")
```

```
uncouple_stats <- lm(leaching_kg_ha ~ drainage_mm * treatment, data = plot_hydrology)
```

```
print("--- UNCOUPLING STATISTICS (ANCOVA) ---")
```

```
[1] "--- UNCOUPLING STATISTICS (ANCOVA) ---"
```

```
print(anova(uncouple_stats))
```

Analysis of Variance Table

Response: leaching_kg_ha

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
drainage_mm	1	827	827	65.91	4.6e-08 ***
treatment	3	147	49	3.90	0.022 *
drainage_mm:treatment	3	141	47	3.76	0.026 *
Residuals	22	276	13		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

detailed coefficients to see specific slope differences

```
print(summary(uncouple_stats))
```

Call:

```
lm(formula = leaching_kg_ha ~ drainage_mm * treatment, data = plot_hydrology)
```

Residuals:

Min	1Q	Median	3Q	Max
-9.451	-0.920	-0.065	0.728	12.134

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.12654	1.54550	0.08	0.935
drainage_mm	0.00367	0.01689	0.22	0.830
treatmentCorn	3.74118	2.57728	1.45	0.161
treatmentSorghum	0.15045	2.30473	0.07	0.949
treatmentSoy	1.30385	2.44335	0.53	0.599
drainage_mm:treatmentCorn	0.02926	0.02534	1.15	0.261
drainage_mm:treatmentSorghum	0.05494	0.02839	1.94	0.066 .
drainage_mm:treatmentSoy	0.06290	0.01956	3.22	0.004 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.54 on 22 degrees of freedom

Multiple R-squared: 0.802, Adjusted R-squared: 0.738

F-statistic: 12.7 on 7 and 22 DF, p-value: 1.98e-06

14 Table S2: Cover Crop Biomass

```
library(readxl)
library(gt)

# Load rye cover crop biomass data
rye_biomass <- bind_rows(

  read_excel("data/BiomassData_Rye_20230522.xlsx") |> mutate(year = 2023),
  read_excel("data/BiomassData_Rye_20240515.xlsx") |> mutate(year = 2024)
) |>
mutate(
  year = factor(year, levels = c(2023, 2024)),
  Plot = as.character(`Plot ID`),
  biomass_mg_ha = (`Dry Mass + Dry Bag Mass` - `Dry Bag Mass`) * 0.01
) |>
select(year, Plot, biomass_mg_ha)

# Summary stats by year
rye_biomass_summary <- rye_biomass |>
group_by(year) |>
summarise(
  n = n(),
  mean = mean(biomass_mg_ha, na.rm = TRUE),
  sd = sd(biomass_mg_ha, na.rm = TRUE),
  se = sd / sqrt(n),
  min = min(biomass_mg_ha, na.rm = TRUE),
  max = max(biomass_mg_ha, na.rm = TRUE),
  .groups = "drop"
)

# Publication table
```

Table S2. Cereal Rye (Cover Crop) Biomass
Biomass in Mg ha⁻¹, measured at termination

Year	n	Mean	SD	SE	Min	Max
2023	16	1.58	0.29	0.07	0.97	2.01
2024	16	1.65	0.53	0.13	1.01	2.98

```
table_s2 <- rye_biomass_summary |>
  gt() |>
  cols_label(
    year = "Year",
    n     = "n",
    mean  = "Mean",
    sd    = "SD",
    se    = "SE",
    min   = "Min",
    max   = "Max"
  ) |>
  fmt_number(columns = c(mean, sd, se, min, max), decimals = 2) |>
  tab_header(
    title = md("**Table S2. Cereal Rye (Cover Crop) Biomass**"),
    subtitle = "Biomass in Mg ha-1, measured at termination"
  ) |>
  cols_align(aligned = "center", columns = everything())

table_s2

gtsave(table_s2, "SubmissionFiguresTables/Table_S2_Cover_Crop_Biomass.rtf")
```

15 Save Tables and Figures

Save to SubmissionFiguresTables

```
# Create output directory

# --- FIGURES ---
# Figure 1: Weather
ggsave(
  "SubmissionFiguresTables/Figure1_Weather.png",
  plot = six_panel_plot,
  width = 84, height = 105, units = "mm", dpi = 600, bg = "white"
)

# Figure 2: Soil Moisture & Temperature
ggsave(
  "SubmissionFiguresTables/Figure2_Soil_Moisture_Temperature.png",
  plot = soil_4panel,
  width = 174, height = 174, units = "mm", dpi = 600, bg = "white"
)

# Figure 3: N2O Flux & Precipitation (Dual Axis)
```

```

ggsave(
  "SubmissionFiguresTables/Figure3_N20_Flux_Precipitation.png",
  plot = figure3_dual_axis,
  width = 174, height = 174, units = "mm", dpi = 600, bg = "white"
)

# Figure 4: Leaching & Precipitation
ggsave(
  "SubmissionFiguresTables/Figure4_Leaching_Precipitation.png",
  plot = fig_leaching,
  width = 174, height = 174, units = "mm", dpi = 600, bg = "white"
)

# Figure 5: Nitrate Speciation
ggsave(
  "SubmissionFiguresTables/SuppFigurex_Nitrate_Concentration_Proportion.png",
  plot = fig_nh4no3,
  width = 174, height = 200, units = "mm", dpi = 600, bg = "white"
)

# Figure 6: Hydrology
ggsave(
  "SubmissionFiguresTables/Figure6_Hydrology.png",
  plot = p_hydro,
  width = 174, height = 87, units = "mm", dpi = 600, bg = "white"
)

# Supp Figure row vs interrow
ggsave(
  "SubmissionFiguresTables/rowvsinterrow.png",
  plot = rowvsinterrow,
  width = 174, height = 200, units = "mm", dpi = 600, bg = "white"
)

# --- TABLES ---
# Table 1: Daily Flux Statistics (Supplementary)
gtsave(
  supp_table1,
  "SubmissionFiguresTables/Table_S1_Daily_Flux_Statistics.rtf"
)

# Table 2: Validation (Cumulative N20)
gtsave(
  validation_table,
  "SubmissionFiguresTables/Table_S2_Validation_Cumulative_N20.png"
)

# Table 3: Annual Nitrogen Losses & Environmental Costs
gtsave(
  ft,
  "SubmissionFiguresTables/Table3_Annual_Nitrogen_Losses.rtf"
)

```

```
cat("\n All figures and tables saved to SubmissionFiguresTables/\n")
```

All figures and tables saved to SubmissionFiguresTables/